PROJECT FINAL REPORT

GROUP PROJECT

IIT271-2

Group No: IIT-18

UniAlloc

Human Resource Allocation System For University Office

Industrial Information Technology Degree Program

Department of Computer Science and Informatics

Faculty of Applied Sciences

Uva Wellassa University of Sri Lanka

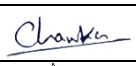
2026

DECLARATION

We hereby declare that the project has been developed by us and is our own work and has not been submitted in any form for another degree, diploma, or certificate at any university or other institution of tertiary education, and no part has been plagiarized without citations under the supervision of Dr. N. P. Samarasinghe, Senior Lecturer, Department of Computer Science and Informatics, Faculty of Applied Sciences, Uva Wellassa University, Badulla, Sri Lanka. This Project Report is submitted for the partial fulfillment of the requirement of the course unit IIT271-2 for the degree of Industrial Information Technology Degree Program.

GROUP DETAILS

Group No: IIT-18

No.	Name of the Student	Index Number	E-mail address	Signature
1	Mr. Suhan W.H.	UWU/IIT/23/011	iit23011@std.uwu.ac.lk	
2	Mr. Sandeep G.C.	UWU/IIT/23/020	iit23020@std.uwu.ac.lk	
3	Mr. Chathuranga D.L.	UWU/IIT/23/030	iit23030@std.uwu.ac.lk	
4	Mr. Arachchi D.C.K.	UWU/IIT/23/024	iit23024@std.uwu.ac.lk	

SUPERVISOR DETAILS

Name of the Supervisor	E-mail	Contact Number
Dr. N. P. Samarasinghe	nayanathara@uwu.ac.lk	+94 553560090

Name of the Co-Supervisor	E-mail	Contact Number
Mr. K.B.L. Jayawickrama	kavindulakshan2980@gmail.com	+94 704512641

Project Approval:

…….. ..……

Dr. N. P. Samarasinghe Mr. K.B.L Jayawickrama

Date: 17.09.2026 Date: 17.09.2026

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our supervisor, Dr. N. P. Samarasinghe, and co-supervisor, Mr. K.B.L. Jayawickrama, for their invaluable guidance, continuous support, and constructive feedback throughout the development of this project. We are also grateful to the Department of Computer Science and Informatics, Faculty of Applied Sciences, Uva Wellassa University of Sri Lanka, for providing the resources and environment necessary to complete this work. Finally, we extend our heartfelt thanks to our families and friends for their encouragement and support throughout this project.

ABSTRACT

University Dean's Offices and Departments commonly manage lecturer workloads and student supervision assignments using spreadsheets, informal messaging, and personal judgement. This approach provides limited visibility into lecturer availability, leads to inconsistent allocation decisions, duplicated commitments, and makes it difficult to track progress across multiple departments and faculties. UniAlloc is a web-based Human Resource Allocation System developed to address this problem by replacing ad-hoc workload management with a transparent, role-based platform.

The system supports five user roles System Admin, Dean, Department Head, Lecturer, and Student each with access scoped to their position within a two-tier Faculty-Department structure. Deans and Department Heads create assignments with priority, deadlines, and estimated hours, and the system displays each lecturer's real-time capacity (assigned hours versus available hours) before an assignment is made, flagging overload conditions automatically. When an assignment requires a lecturer outside the assigner's authority scope, a formal cross-department or cross-faculty request workflow routes the request for review and approval. Lecturers update the progress of assigned work with timestamped notes, and students submit supervisor requests through a dedicated portal that is reviewed and actioned by the Dean. Notifications and audit logs keep users informed and provide a record of key actions.

The system was analysed and designed using standard object-oriented techniques, including class, entity-relationship, and use case diagrams, and was implemented as a web application with a PHP and MySQL back end and a Next.js front end, following the Waterfall development process. Each module authentication and account management, faculty/department management, assignment and capacity management, cross-boundary requests, progress tracking, and student supervisor requests was implemented and unit tested individually before integration testing confirmed correct communication between the front end and back end.

The results indicate that UniAlloc successfully replaces manual, spreadsheet-based allocation with a centralized, transparent process that gives Deans and Department Heads real-time visibility into lecturer workload, formalizes cross-boundary allocation requests, and gives students a reliable channel for requesting supervision, improving fairness, accountability, and decision-making in academic human resource allocation.

TABLE OF CONTENT

DECLARATION i
GROUP DETAILS i
SUPERVISOR DETAILS i
ACKNOWLEDGEMENTS ii
ABSTRACT iii
TABLE OF CONTENT iv
LIST OF FIGURES vi
LIST OF TABLES vii
CHAPTER 1: INTRODUCTION 1
1.1 Title of the Project 1
1.2 Project Description 1
1.3 Background and Motivation 2
1.4 Problem in Brief 2
1.5 Proposed Solution 2
1.6 Project Aim and Objectives 4
1.7 Significance of the Study 4
CHAPTER 2: METHODOLOGY 6
2.1 Introduction 6
2.2 Requirements Identification 7
2.2.1 Functional Requirements 7
2.2.2 Non-Functional Requirements 8
2.2.3 User Roles 10
2.2.4 System Requirements (Hardware/Software) 11
2.3 System Design (Architecture, Modules, Data Flow, Security, UI/UX) 12
2.3.1 System Diagram 12
2.3.1.1 Class Diagram 12
2.3.1.2 ER Diagram 13
2.3.1.3 Use Case Diagram 14
2.3.2 System Architecture 15

2.3.3 UI/UX Design Philosophy 17
2.4 Deployment 18
2.5 Maintenance 19
CHAPTER 3: IMPLEMENTATION 20
3.1 Introduction 20
3.2 Module Implementation 20
3.3 Integration of Modules 28
CHAPTER 4: TESTING AND EVALUATION 29
4.1 Introduction 29
4.2 Testing 29
4.3 Evaluation 29
CHAPTER 5: CONCLUSION 32
5.1 Conclusion 32
5.2 Project Plan (Gantt Chart) 33
5.3 Individual Contribution 34
References 35
Appendixes 36

LIST OF FIGURES

Figure 01: UniAlloc Platform

Figure 02: Waterfall Model

Figure 03: UniAlloc Platform Class Diagram

Figure 04: UniAlloc Platform ER Diagram

Figure 05: UniAlloc Platform Use Case Diagram

Figure 06: Three-Tier Architecture

Figure 07: UI/UX Concept

Figure 08: Maintenance

Figure 09: Authentication and Account Management Flowchart

Figure 10: Faculty and Department Management Flowchart

Figure 11: Assignment Management and Capacity Tracking Flowchart

Figure 12: Cross-Boundary Request and Workload Appeal Flowchart

Figure 13: Student Supervisor Request and Progress Tracking Flowchart

Figure 14: Notifications and Audit Logging Flowchart

Figure 15: Evaluation Results by Module

LIST OF TABLES

Table 1: Evaluation Results

Table 2: Project Plan (Gantt Chart)

Table 3: Individual Contribution

Table 4: Sample Test Cases

CHAPTER 1: INTRODUCTION

1.1 Title of the Project

UniAlloc Human Resource Allocation System For University Office

1.2 Project Description

UniAlloc is a web-based Human Resource Allocation System designed for a university Dean's Office. It replaces ad-hoc, spreadsheet-driven workload management with a transparent, role-based platform that enables Deans and Department Heads to allocate assignments based on real-time lecturer capacity, supports cross-department and cross-faculty request workflows, and provides a formal portal for students to request project/thesis supervisors with trackable status updates.

This system focuses on allocating and tracking academic work across a two-tier university structure (Faculties containing Departments). It supports assignment creation, workload/capacity tracking, progress updates, cross-boundary request workflows, and student supervision requests. This report covers the requirements analysis, design, implementation, and evaluation of the completed web-based system.

Project Boundaries:

- Role-based access for System Admin, Dean, Department Head, Lecturer, and Student
- Faculty/Department management (faculty contains departments; users belong to departments)
- Assignment management (create, prioritize, estimate hours, set deadlines, assign, track status)
- Real-time workload capacity view (assigned hours vs. available hours) and overload warnings
- Cross-department and cross-faculty request workflows for out-of-scope allocations
- Progress tracking (percentage updates with timestamped notes)
- Student supervisor request portal with status tracking and Dean review/assignment
- Notifications and audit logs for critical system actions

1.3 Background and Motivation

University Dean's Offices and Departments often manage lecturer workloads and supervision assignments using spreadsheets, informal messaging, and personal judgement. This leads to limited visibility into who is available, inconsistent allocation of decisions, duplicated commitments, and difficulty tracking progress across multiple departments and faculties.

This project is motivated by the need for a centralized, hierarchy-aware platform that provides real-time capacity information before assignments are made, supports formal cross-department/cross-faculty request workflows, and maintains a clear audit trail of who assigned what work and when. By combining allocation, monitoring, and structured requests in one system, UniAlloc aims to improve fairness, accountability, and decision-making in academic HR management.[1],[2]

1.4 Problem in Brief

- Unbalanced lecturer workloads: Without a centralized capacity view, some lecturers become overloaded while others remain underutilized.
- Informal cross-department assignments: When work needs to cross department/faculty boundaries, there is no formal channel, causing confusion and disputes.
- Unsystematic student project requests: Supervisor requests are often handled informally with little tracking, leading to lost requests and inconsistent outcomes.

1.5 Proposed Solution

UniAlloc provides a centralized platform where Deans and Department Heads create assignments and allocate them to lecturers based on real-time capacity information (assigned hours vs. available hours). When a required lecturer is outside the assigner's authority scope, the system supports formal cross-department or cross-faculty requests that are tracked to completion. Lecturers update progress on assigned work, and students submit supervisor requests through a structured portal that routes to the Dean for review and assignment.[2]

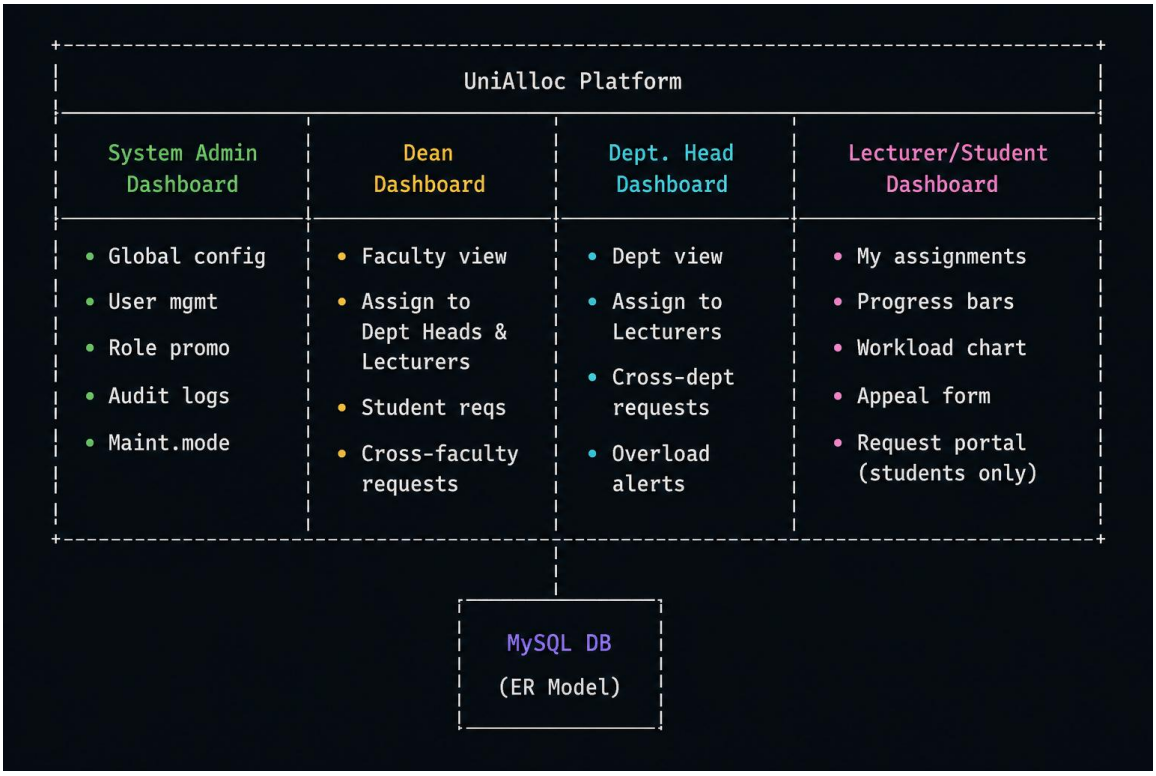


Figure 01: UniAlloc Platform

How it works:

1. Deans and Department Heads create assignments with priority, deadline, and estimated hours.
2. Before assigning, the system shows each lecturer's capacity: assigned hours vs. available hours.
3. If the needed lecturer is out of scope, the assigner submits a cross-department or cross-faculty request and tracks its status.
4. Lecturers update progress (0–100%) with timestamped notes; dashboards reflect live status.
5. Students submit supervisor requests; the Dean reviews and assigns an appropriate lecturer/department head.
6. Overload detection triggers notifications and helps supervisors identify underloaded alternatives.

1.6 Project Aim and Objectives

Aim

To design and implement a role-based, hierarchy-aware web system for a university faculty Office that enables fair and transparent lecturer workload and supervision allocation through real-time capacity visibility, structured request workflows, and progress tracking.

Objectives

- Implement five-level role-based access control (System Admin, Dean, Department Head, Lecturer, Student).
- Provide assignment management with priority, deadlines, and estimated hours.
- Implement real-time workload capacity tracking (assigned hours vs. available hours) with overload alerts.
- Support cross-department and cross-faculty request workflows for out-of-scope allocations.
- Provide per-assignment progress tracking and role-based dashboards for monitoring.
- Provide a student portal for supervisor requests with status tracking and Dean review.
- Ensure that the system is secure, usable, maintainable, and scalable for institutional use.

1.7 Significance of the Study

The UniAlloc system enhances the allocation process of lecturer workload and requests for students' supervisors by automating the manual allocation process from spreadsheets and informal communications to a single, centralized web-based system. It offers real-time workload visibility to give Deans and Department Heads a clear understanding of workloads so that tasks can be fairly allocated and lecturers are not overworked.

The system also enables the introduction of structured workflows of requests from departments and faculties, ensuring transparency and accountability in the allocation

process. Furthermore, students can make and monitor requests from supervisors via a formalized entry point, which eliminates delays and helps with communication.

In addition to existing manual techniques, UniAlloc provides role-based access control, workload monitoring, progress tracking and notifications, as well as audit logging. These are all features that help to make operations more efficient, inform decision-making and assist to a more organized and reliable HR allocation process within the University.

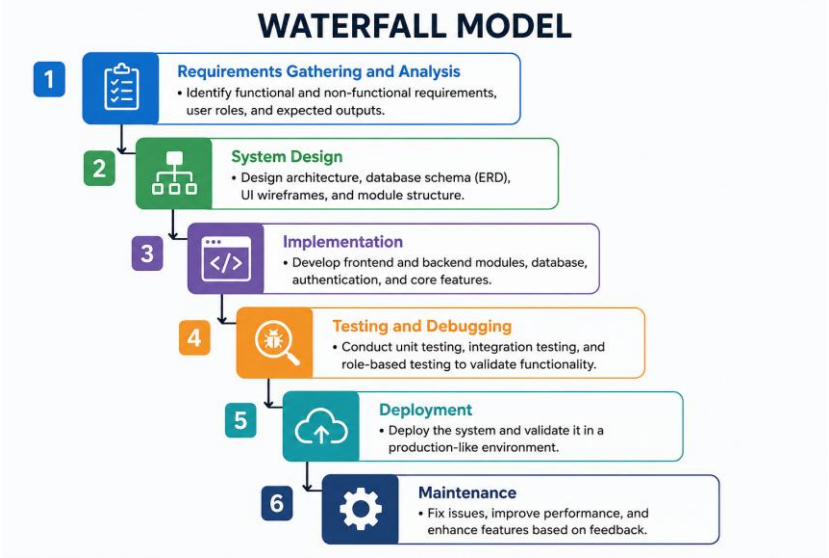
CHAPTER 2: METHODOLOGY

This project follows the Waterfall Software Development Methodology. Waterfall is a sequential approach where each phase is completed before moving to the next phase. This methodology is suitable when requirements are relatively well understood, and the project requires structured documentation and clear deliverables.[3],[4]

2.1 Introduction

This project followed the Waterfall model of the Software Development Life Cycle (SDLC), a linear and sequential process in which each phase is completed before the next begins. This approach was selected because the project requirements were well understood from the outset and the team benefited from a structured, easily trackable development process across the six phases described below (Figure 1).

1. Requirements Gathering and Analysis: Identify functional and non-functional requirements, user roles, and expected outputs.
2. System Design: Design architecture, database schema (ERD), UI wireframes, and module structure.
3. Implementation: Develop frontend and backend modules, database, authentication, and core features.
4. Testing and Debugging: Conduct unit testing, integration testing, and role-based testing to validate functionality.
5. Deployment: Deploy the system and validate it in a production-like environment.
6. Maintenance: Fix issues, improve performance, and enhance features based on feedback.



Figures 02: Waterfall Model

2.2 Requirements Identification

2.2.1 Functional Requirements

- Authentication and Authorization: The system shall provide secure login and enforce role-based access control (System Admin, Dean, Department Head, Lecturer, Student).
- Faculty/Department and User Management: Authorized roles shall be able to manage faculty/department structures and create/manage users within their scope.
- Assignment Management: Deans and Department Heads shall be able to create assignments with priority, deadlines, and estimated hours and assign them within their authority scope.
- Capacity Tracking and Overload Detection: The system shall calculate assigned hours vs. available hours for lecturers and warn/flag overload conditions before and after assignment.
- Cross-Boundary Request Workflows: The system shall support cross-department and cross-faculty requests when an assigner needs a lecturer outside their scope, including approval/rejection and status tracking.
- Progress Tracking: Lecturers shall be able to update assignment progress (0–100%) with timestamped notes; supervisors shall view progress in dashboards.

- Student Supervisor Requests: Students shall be able to submit supervision requests; Deans, and Department Heads within their own department, shall review and assign a supervisor; students shall track request status.
- Notifications and Audit Logging: The system shall provide in-app notifications and maintain audit logs for critical actions and workflow events.
- Workload appeals: Lecturers shall be able to submit workload appeals with justification; Department Heads/Deans shall review and resolve them.
- Role promotion workflows: The system shall support controlled promotions with appropriate approvals.
- Assignment filtering and search: Users and supervisors shall be able to search and filter assignments by status, priority, deadlines, department, and assignee.
- Workload overview: Deans and Department Heads shall view summarized capacity/overload information to support fair allocation decisions.
- Leadership Vacancy Detection: The system shall detect faculties without an active Dean and departments without an active Head and surface these as dashboard alerts to the System Admin.

2.2.2 Non-Functional Requirements

- Security: Passwords shall be hashed; authentication shall use secure tokens; HTTPS shall be enforced; input validation and parameterized queries shall reduce injection risks.
- Usability: The interface shall be intuitive and role-specific, reflecting the university hierarchy and enabling key tasks with minimal steps.
- Performance: Dashboards should load quickly; APIs should use pagination and indexing to support large numbers of users and assignments.
- Compatibility: The system shall work on modern browsers (Chrome, Firefox, Safari, Edge) and be responsive across common screen sizes.
- Maintainability: The codebase shall be modular and documented to support future enhancements and maintenance.

- Availability: The system should be available at least 99.5% of the time during working days, excluding scheduled maintenance, to support continuous office operations.
- Recoverability (Backup & Restore): The system should support automated database backups and allow restoration within a defined recovery time objective to prevent data loss.
- Reliability: The system should handle failures gracefully (e.g., meaningful error messages) and protect data consistency for task updates.

2.2.3 User Roles

System Admin

- Manage global configuration, users, and roles across all faculties
- Approve and confirm promotions (e.g., Department Head → Dean) where required
- View audit logs and maintain the overall system (including maintenance mode)

Dean

- View faculty-wide capacity and workload overview across departments
- Assign work to Department Heads and Lecturers within the faculty
- Review student supervisor requests, forward department and assign a supervisor
- Handle cross-faculty requests (approve/reject) and manage overload alerts
- View and manage tasks assigned to them personally through the "My Work" view, reflecting that Deans can also be assignees, not just assigners

Department Head

- Assign work to lecturers within their department and monitor progress
- View department workload/capacity and receive overload notifications
- Submit cross-department requests when needing a lecturer outside the department
- Review workload appeals and escalates/coordinate with the Dean when needed
- Review and manage student supervisor requests scoped to their own department
- View and manage tasks assigned to them personally through the "My Work" view, reflecting that Department Heads can also be assignees, not just assigners

Lecturer

- View assigned work, deadlines, and personal workload/capacity summary
- Update assignment progress (0–100%) and add timestamped progress notes
- Submit work requests or workload appeals through formal workflows

Student

- Submit a formal request for a project/thesis/research supervisor
  Track the status of submitted requests (e.g., Pending / Approved / Rejected / Assigned)

2.2.4 System Requirements (Hardware/Software)

Software Requirements

- Updated web browser (Chrome, Edge, Firefox, Safari)
- Supported operating system (Windows, macOS, Linux)
- Web server environment: Apache (recommended) or Nginx
- Server-side runtime: PHP 8.x with PDO extension enabled
- Database system: MySQL 8.x (or compatible MariaDB)
- Node.js 18+ and npm/pnpm (required for Next.js frontend build)
- Package and dependency management: Composer (PHP) and npm/pnpm (JavaScript)

Development Tools

- Code editor / IDE: Visual Studio Code
- Version control: Git with GitHub
- UI/UX design tool: Figma (wireframes, mockups, prototypes)
- API testing tool: Postman or Insomnia
- Database client/administration: phpMyAdmin, MySQL Workbench
- Collaboration/communication (optional): Microsoft Teams / Discord
- Documentation (optional): Microsoft Word / Google Docs

Hardware Requirements

- Stable internet connection
- Laptop/Desktop computer
- Minimum 4GB RAM (8GB recommended)
- Minimum 10GB free disk space
- Dual-core processor or higher

2.3 System Design (Architecture, Modules, Data Flow, Security, UI/UX)

During the System Design phase, the project defines the overall solution blueprint, including the application architecture, major modules, high-level data flow, security controls, and UI/UX approach. This ensures the implementation phase proceeds with clear technical guidance and consistent design decisions.[3],[5],[6]

2.3.1 System Diagram

2.3.1.1 Class Diagram

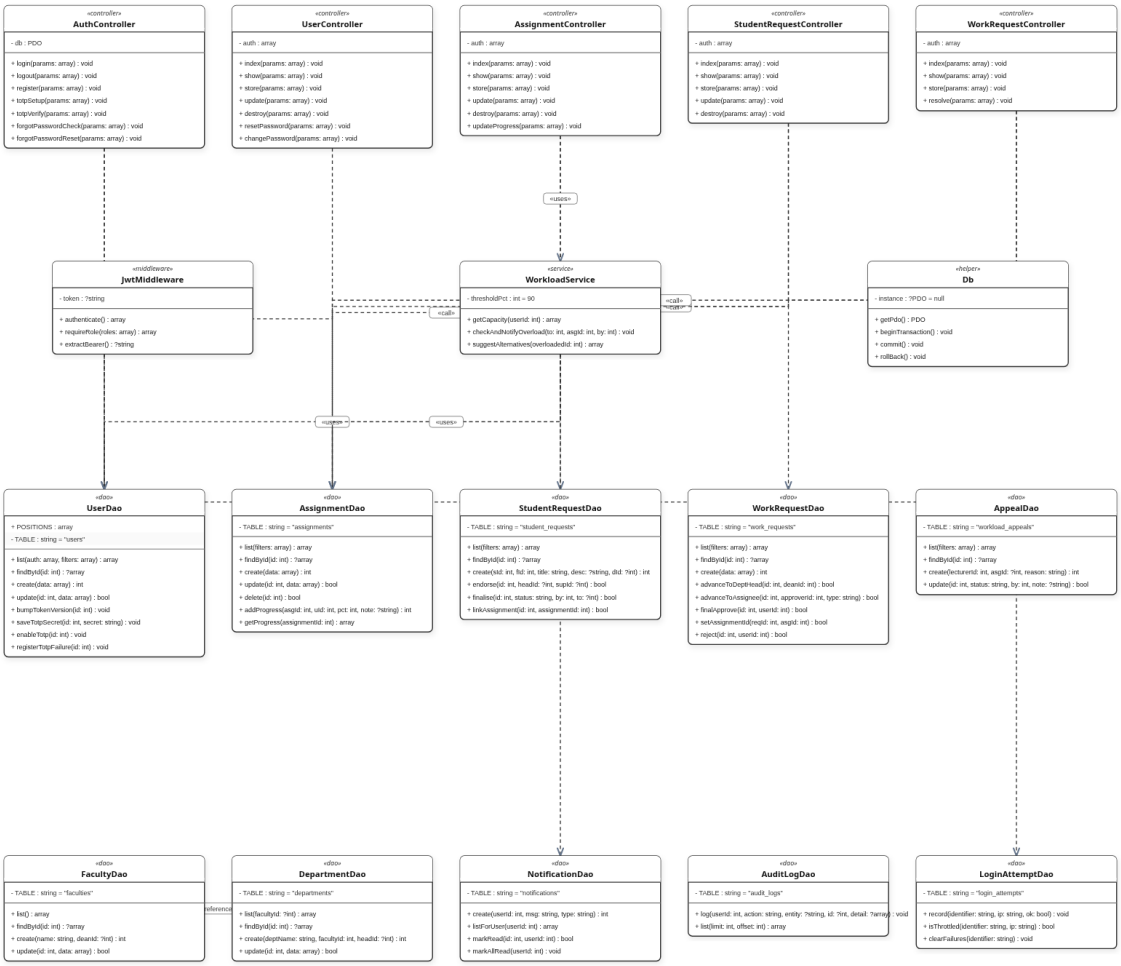


Figure 03: UniAlloc Platform Class Diagram

2.3.1.2 ER Diagram

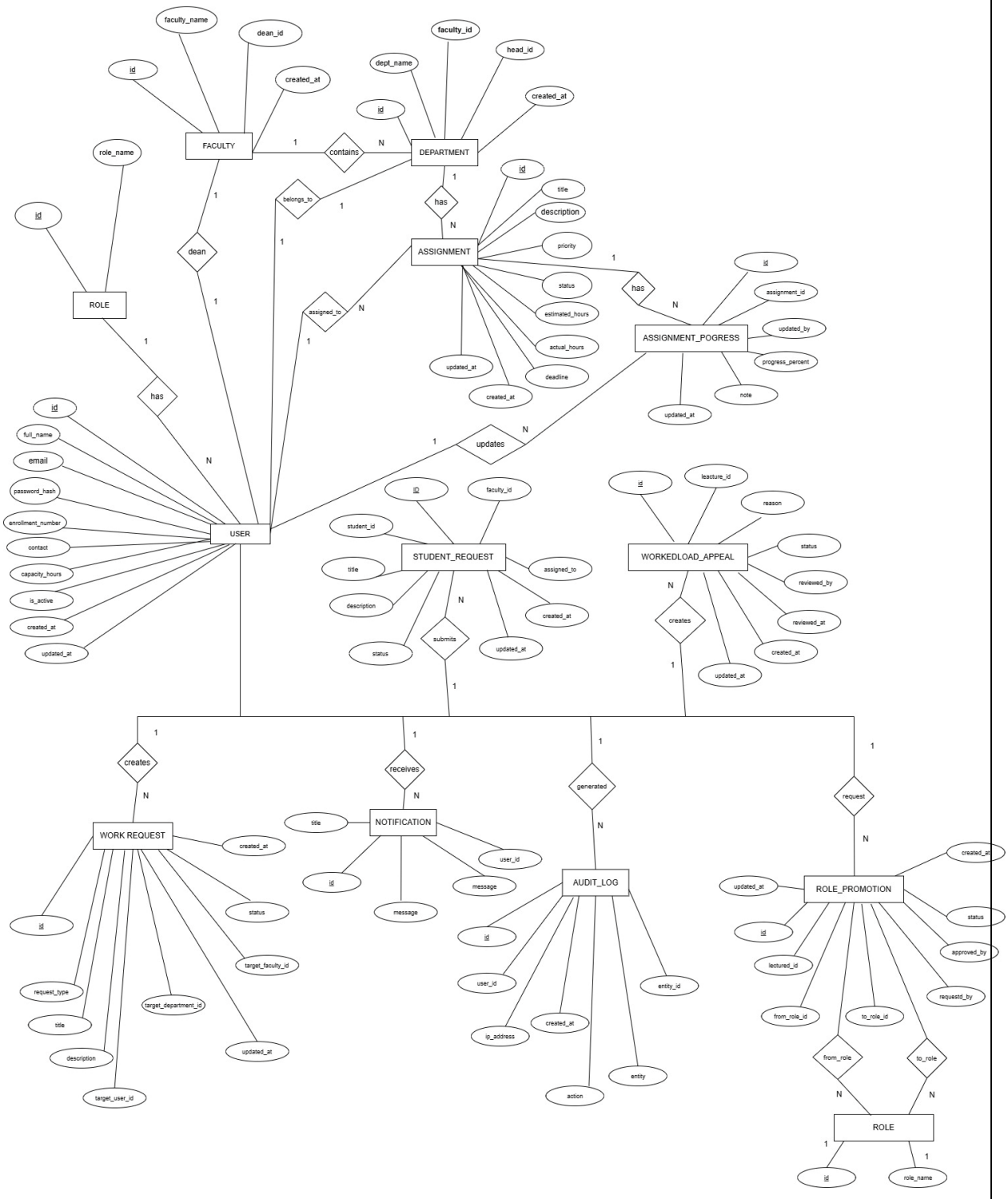


Figure 04: UniAlloc Platform ER Diagram

2.3.1.3 Use Case Diagram

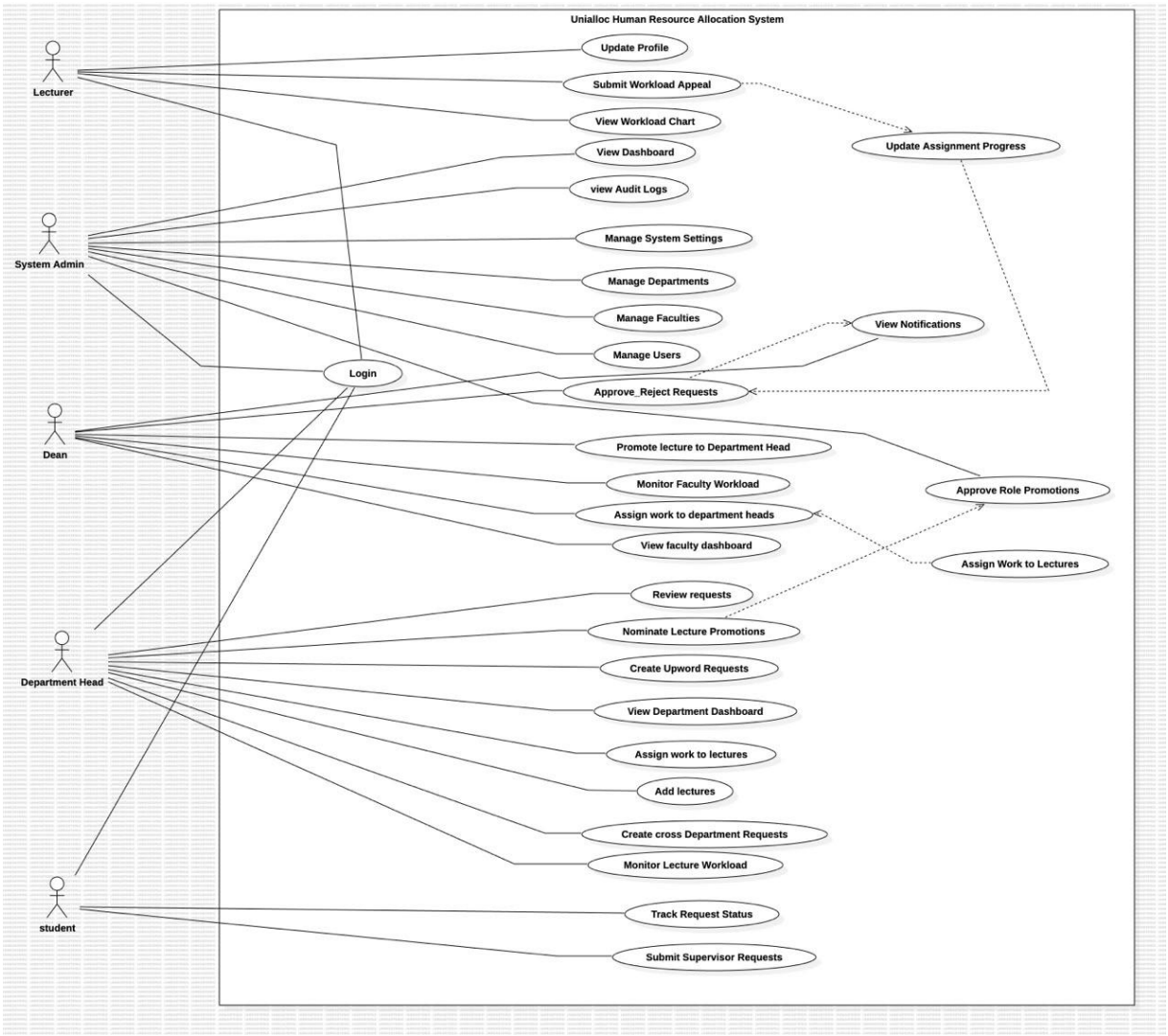


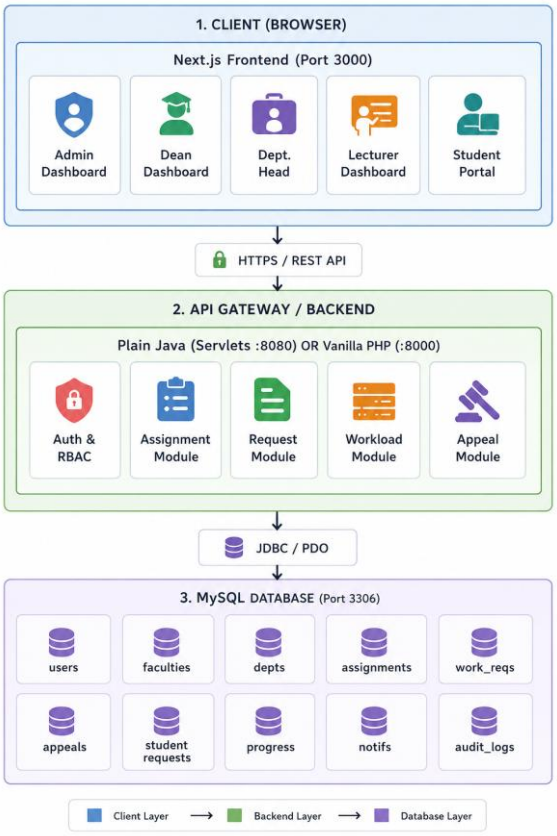
Figure 05: UniAlloc Platform UseCase

2.3.2 System Architecture

The system is designed using a standard three-tier architecture consisting of a client layer (web browser/Next.js web app), application layer (pure REST API built with Vanilla PHP), and data layer (MySQL database). The frontend provides role-based dashboards and user interfaces, while the backend exposes JSON-only endpoints for authentication, role-based access control, and business logic for assignments, capacity, requests, notifications, and reporting. The backend communicates with MySQL using PDO with parameterized queries and returns consistent JSON responses with appropriate HTTP status codes.

Major System Modules

- Authentication & RBAC Module: handles login, token/session validation, and strict role/scope enforcement.
- User, Faculty & Department Module: manages faculties, departments, users, capacity hours, and authority scope.
- Assignment Module: assignment creation, prioritization, deadlines/effort hours, and status tracking.
- Capacity & Overload Module: calculates assigned vs. available hours and triggers overload alerts.
- Request & Appeal Module: cross-department/cross-faculty requests and workload appeals workflows.
- Student Request Module: supervision request submission, Dean review, assignment, and status tracking.
- Notifications Module: alerts for assignments, deadlines, overload conditions, and request status changes.
- Audit Logging Module: records critical actions for accountability and traceability.
- Leadership Vacancy Alert Module: detects a faculty with no active Dean or a department with no active Head (including cases where the incumbent is marked "On Study Leave") and surfaces it as a dashboard alert.
- System Settings Module: allows the System Admin to configure global, system-wide application settings.



Figures 06: Three-Tier Architecture

High-Level Data Flow (Summary)

1. A user (System Admin/Dean/Department Head/Lecturer/Student) authenticates to the system and receives a secure session/token.
2. System Admin manages global settings; Deans manage faculty-scoped users and structure as permitted.
3. Deans and Department Heads create assignments (priority, deadline, estimated hours) within their authority scope.
4. Before assigning, the backend calculates lecturer capacity (assigned hours vs. available hours) and flags overload risk.
5. If the target lecturer is outside scope, the assigner submits a cross-department/cross-faculty request; the approver reviews and updates status.
6. Allocated lecturers update progress (0–100%) with timestamped notes; updates are stored in the progress log.
7. Students submit supervisor requests; requests route to the Dean for review/assignment, and students can track status changes.
8. Notifications are generated for key events, and critical actions are recorded in audit logs.

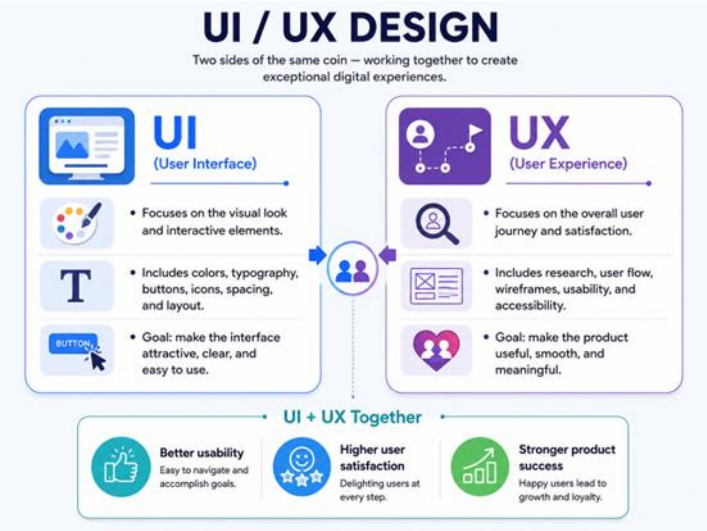
9. Role-based dashboards aggregate workload, capacity, progress, and pending requests for monitoring and decision-making.

Security Considerations (Design Summary)

- Authentication: Use token-based authentication (e.g., JWT) with expiry and secure credential handling.
- Authorization: Enforce role-based access checks on every protected action to prevent broken access control.
- Input validation: Validate/sanitize input and use parameterized queries to reduce injection risks.
- Logging and monitoring: Maintain audit logs for key actions (logins, assignments, approvals, role changes).
- Secure configuration: Apply secure defaults and keep components updated, aligning with OWASP Top 10 guidance.

2.3.3 UI/UX Design Philosophy

The user interface is designed to be clean and task-focused, allowing users to quickly understand workload, task status, and progress. The design emphasizes readability, consistency, and role-based visibility of functions. [6],[7],[8]



Figures 07: UI/UX Concept

- Dark-first visual theme to reduce eye strain and make dashboards comfortable for long sessions.
- Glass morphism-inspired surfaces (soft blur/translucent cards) to separate content layers while keeping the UI modern.

- Gradient accent colors for emphasis (primary actions, status highlights, and key metrics) without overwhelming core data.
- Micro-animations for feedback and clarity (hover states, transitions, progress changes) where applicable.
- Consistent layout, labels, and navigation across role-based dashboards to reduce learning effort.
- Responsive design for desktop and mobile devices, maintaining accessibility (contrast, readable sizes, and clear status indicators).

2.4 Deployment

Deployment involves preparing the system to run in a production-like environment. The application is configured on a web server with the required runtime and database, then verified through smoke tests to ensure that key functions work correctly after deployment.[9]

Key activities:

- Configure server environment (Apache/Nginx, PHP, MySQL, Node.js for frontend build).
- Set up environment variables (database credentials, secret keys, allowed origins).
- Deploy backend APIs and import database schema.
- Build and deploy frontend application bundle.
- Configure security settings (HTTPS, secure headers, access rules).
- Conduct smoke testing and verify login, dashboards, and workflows.
- Create backup/restore procedures for database.

Deliverables:

- Deployed prototype system
- Deployment guide (setup steps, configuration values, troubleshooting tips)
- Verified working environment and baseline backup

2.5 Maintenance

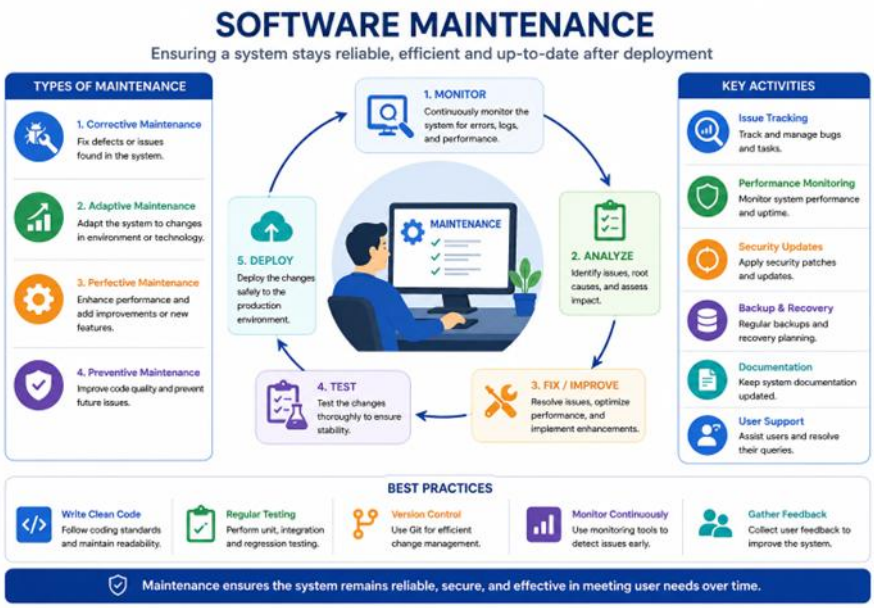
Maintenance begins after deployment and continues throughout the system's usage. The focus is on resolving issues, improving performance, refining usability, and implementing enhancements based on feedback. This phase ensures the system remains stable, secure, and aligned with evolving university needs.[9]

Key activities:

- Monitor system logs and address runtime errors or user-reported issues.
- Apply security patches and dependency updates when required.
- Optimize database queries and improve response times as usage grows.
- Improve UI/UX based on user feedback (navigation, forms, dashboards).
- Add small enhancements such as new reports, filters, or notifications.
- Perform periodic backups and validate restore procedures.

Deliverables:

- Maintenance log (bugs fixed, updates applied)
- Minor feature enhancements and UI refinements
- Updated system documentation and release notes



Figures 08: Maintenance

CHAPTER 3: IMPLEMENTATION

3.1 Introduction

This chapter documents the implementation of UniAlloc's core modules, following the design produced in Chapter 2. Each module is described in terms of its purpose, inputs, processing logic, and outputs, together with the pseudocode used to implement its core logic. Screenshots of the working modules are provided in the Appendixes.

3.2 Module Implementation

Module: Authentication and Account Management

Purpose: Allow all five user roles to securely register and log in, and allow the System Admin to manage the account lifecycle, including activation, deactivation, and role promotion.

Inputs: Username/email, password, role, admin actions (activate, deactivate, promote).

Process:

- Validate credentials against the stored hashed password on login.
- Issue a session scoped to the user's role and assigned faculty/department.
- System Admin creates accounts for Deans, Department Heads, and Lecturers; students self-register.
- Admin promotes a Lecturer to Department Head or deactivates an account.
- Enforce role-based access control on every subsequent request.

Outputs: An authenticated session or an error message; an updated account list showing each user's current role and status.

Pseudo Code:

```
Begin
  Read username and password
  If credentials valid Then
    Load user role and assigned faculty/department
```

Create session scoped to role

Redirect to role-specific dashboard

Else

Show “Invalid credentials” error

End If

If Admin requests Promote/Deactivate Then

Validate admin permission

Update user role or status in database

Log action to audit trail

End If

End

Figure 09: Authentication and Account Management Flowchart

Module: Faculty and Department Management

Purpose: Allow the System Admin to maintain the two-tier Faculty-Department structure that scopes every other module in the system.

Inputs: Faculty name, department name, parent faculty, admin actions (add, edit, delete).

Process:

- Display existing faculties and their departments in a hierarchical list.
- Admin adds a faculty, then adds departments under it.
- Validate that a department name is unique within its faculty before saving.
- Admin edits or deletes a faculty/department, with status changes cascading to affected users where required.

Outputs: Success/error messages; an updated faculty-department hierarchy that scopes Deans, Department Heads, and Lecturers elsewhere in the system.

Pseudo Code:

Begin

If Add Faculty/Department Then

Validate name uniqueness within scope

Insert record into database

Show success or error message

Else If Edit/Delete Then

Validate no active assignments block the change

Update or remove record

Show success or error message

Else

Display faculty-department hierarchy

End If

End

Figure 10: Faculty and Department Management Flowchart

Module: Assignment Management and Capacity Tracking

Purpose: Let Deans and Department Heads create work assignments for Lecturers while tracking each Lecturer's real-time capacity and flagging overload before an assignment is confirmed.

Inputs: Assignment title, description, priority, deadline, estimated hours, assigned Lecturer.

Process:

- Calculate the target Lecturer's current assigned hours against their available capacity before the assignment is confirmed.
- If assigning the task would exceed the Lecturer's capacity, flag an overload warning to the assigner.

- Save the assignment with its priority and deadline, and update the Lecturer's workload dashboard.
- Provide a workload overview dashboard showing every Lecturer's current load, colour-coded by capacity.

Outputs: A confirmed assignment or an overload warning; an updated workload dashboard and analytics charts.

Pseudo Code:

```
Begin
  Read assignment details and target Lecturer
  Compute currentHours = sum of Lecturer's active assignment hours
  If currentHours + estimatedHours > capacityLimit Then
    Display overload warning to assigner
    If assigner confirms override Then
      Save assignment, mark as overloaded
    End If
  Else
    Save assignment
  End If
  Update workload dashboard and analytics view
End
```

Figure 11: Assignment Management and Capacity Tracking Flowchart

Module: Cross-Boundary Request and Workload Appeal Workflow

Purpose: Formalize the process when an assignment needs a Lecturer outside the assigner's own department or faculty, and let Lecturers appeal an assignment they consider unfair or excessive.

Inputs: Requested Lecturer, target department/faculty, justification, appeal reason.

Process:

- Detect when the requested Lecturer sits outside the assigner's authority scope and route the request to the appropriate Department Head or Dean for approval.
- Approver reviews the request with full context requester, justification, and the target Lecturer's current load and approves, rejects, or forwards it.
- A Lecturer who disagrees with an assignment can submit a workload appeal with a reason; the appeal is routed to the relevant Department Head/Dean.
- Search and filter tools let approvers locate pending requests and appeals by status, department, or date.

Outputs: An approved or rejected request/appeal with a recorded decision and audit entry; an updated request queue.

Pseudo Code:

```
Begin
  If target Lecturer outside assigner's scope Then
    Create cross-boundary request with justification
    Route to Department Head/Dean of target scope
    If approved Then
      Create assignment
    Else
      Notify requester of rejection with reason
    End If
  End If
  If Lecturer submits appeal Then
    Route appeal to relevant approver
    Record decision and update assignment status
  End If
```

End

Figure 12: Cross-Boundary Request and Workload Appeal Flowchart

Module: Student Supervisor Request Portal and Progress Tracking

Purpose: Give students a dedicated channel to request a supervisor, and let Lecturers, Department Heads, and Deans track the progress of assigned work over time.

Inputs: Student details, requested research area/supervisor preference, progress notes, status updates.

Process:

- Student submits a supervisor request with an area of interest and, optionally, a preferred Lecturer.
- Dean reviews pending requests and either assigns a supervisor directly or forwards the request to a Department Head; Department Heads can also directly view and act on requests scoped to their own department.
- Lecturer updates the progress of each assigned task or supervision with a timestamped note and a status marker.
- Progress history is visible to the student, the Lecturer, and the relevant Department Head/Dean.

Outputs: A confirmed supervisor assignment or pending status; an updated progress timeline visible to all stakeholders.

Pseudo Code:

```
Begin
  If Student submits request Then
    Save request with status “Pending”
    Notify Dean
  End If
  If Dean assigns supervisor Then
    Update request status to “Assigned”
```

Notify student and Lecturer

End If

If Lecturer records progress update Then

Append timestamped note and status to progress log

Update visible progress percentage

End If

End

Figure 13: Student Supervisor Request and Progress Tracking Flowchart

Module: Notifications and Audit Logging

Purpose: Keep users informed of events relevant to them and maintain a verifiable record of key actions across the system.

Inputs: System events (assignment created, request approved/rejected, role changed, appeal submitted).

Process:

- Generate an in-app notification whenever an event affecting a user occurs, such as a new assignment, an approval decision, or a progress update.
- Record every state-changing action who, what, and when to an append-only audit log.
- Allow the System Admin to view and filter the audit log by user, module, or date range.

Outputs: Delivered notifications; a queryable audit trail for accountability and troubleshooting.

Pseudo Code:

Begin

On system event:

Identify affected user(s)

Create notification record

Append entry to audit log with actor, action, timestamp

If Admin queries audit log Then

Filter records by user/module/date

Display matching entries

End If

End

Figure 14: Notifications and Audit Logging Flowchart

Module: Leadership Vacancy Alerts

Purpose: Detect when a faculty has no active Dean or a department has no active Head, and surface this as a dashboard alert so the System Admin can act promptly.

Inputs: Current faculty/department leadership assignments; Dean/Department Head account status, including "On Study Leave".

Process:

- Check each faculty for a user actively holding the Dean role, and each department for a user actively holding the Department Head role.
- Treat a leadership account marked "On Study Leave" as inactive for this purpose.
- If no active Dean/Department Head is found, generate a Leadership Vacancy alert on the System Admin dashboard identifying the affected faculty or department.
- Clear the alert once an active Dean/Department Head is reinstated or a replacement is assigned.

Outputs: A dashboard alert listing faculties/departments currently without active leadership.

3.3 Integration of Modules

The seven modules were integrated around a shared authentication layer and the faculty-department hierarchy. Every request first passes through the Authentication and Account Management module, which attaches the caller's role and faculty/department scope; this scope is then used by the Assignment Management, Cross-Boundary Request, and Student Supervisor Request modules to determine what a given user is permitted to see or approve. The Assignment Management module reads live capacity data before creating an assignment, and any resulting event a new assignment, an approval, a rejection, or a progress update is published to the Notifications and Audit Logging module, which is the only module that writes to the audit trail. This event-driven link keeps the modules loosely coupled while ensuring that every state change is both visible to the affected user and recorded for accountability.

CHAPTER 4: TESTING AND EVALUATION

4.1 Introduction

This chapter presents the testing and evaluation of the UniAlloc system. The purpose of testing is to verify that each module functions according to its specified requirements, while evaluation measures whether the completed system meets user expectations and achieves the project's objectives. Both functional testing and a structured user evaluation were carried out on the completed system.

4.2 Testing

4.2.1 Test Strategy

Unit Testing: Each module described in Chapter 3 Authentication and Account Management, Faculty and Department Management, Assignment Management and Capacity Tracking, Cross-Boundary Request and Workload Appeal Workflow, Student Supervisor Request Portal and Progress Tracking, and Notifications and Audit Logging was tested individually against its expected inputs and outputs.

Integration Testing: Modules were tested together to confirm correct communication between the Next.js front end and the PHP/MySQL back end through the API layer, including role-based access control across module boundaries.

Acceptance Testing: Representative end-users from each role Dean, Department Head, Lecturer, and Student exercised the system to validate that it met the intended business requirements and project objectives.

Detailed test cases, including preconditions, steps, expected results, and actual results, are presented in Appendix H – Test Cases.

4.3 Evaluation

4.3.1 Evaluation Design

The evaluation of UniAlloc was conducted to assess its usability, functionality, and overall user satisfaction once the core modules were complete.

Participants: 10 users, comprising 10 Students; the System Admin, Dean, Lecturers and Department Head roles were represented among the evaluation participants.

Method: A questionnaire using a 5-point Likert scale (1 = Very Dissatisfied, 5 = Very Satisfied) was distributed to each participant after they interacted with the system (see Appendix I – User Evaluation Questionnaire).

Focus Areas: The six core modules were evaluated Authentication and Account Management, Faculty and Department Management, Assignment Management and Capacity Tracking, Cross-Boundary Request and Workload Appeal Workflow, Student Supervisor Request Portal, and Notifications and Audit Logging.

4.3.2 Evaluation Results

Module	Very Satisfied	Satisfied	Neutral	Dissatisfied	Very Dissatisfied
Authentication and Account Management	6	3	1	0	0
Faculty and Department Management	5	4	1	0	0
Assignment Management and Capacity Tracking	7	2	1	0	0
Cross-Boundary Request and Workload Appeal Workflow	4	4	2	0	0
Student Supervisor Request Portal	5	3	2	0	0
Notifications and Audit Logging	4	5	1	0	0

Table 1: Evaluation Results

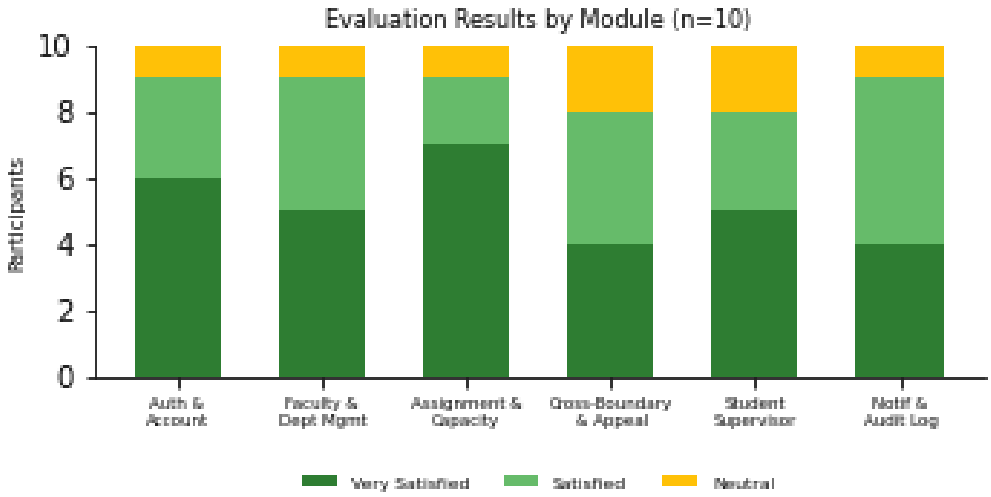


Figure 15: Evaluation Results by Module

Analysis:

A majority of participants reported being either Very Satisfied or Satisfied across all six modules, indicating that the completed system met the expectations of most users. The Assignment Management and Capacity Tracking module received the strongest response, reflecting the value participants placed on real-time visibility into Lecturer workload. The Cross-Boundary Request and Workload Appeal Workflow and the Student Supervisor Request Portal received a comparatively higher share of Neutral ratings, suggesting these newer, less familiar workflows would benefit from clearer in-app guidance and further usability refinement. It should also be noted that the Faculty and Department Management and Cross-Boundary Request and Workload Appeal Workflow modules are primarily used by System Admins, Deans, and Department Heads; since the evaluation pool consisted only of Lecturers and Students, these ratings reflect indirect exposure rather than assessment by the modules' primary intended users, which is a limitation of the current evaluation scope.

CHAPTER 5: CONCLUSION

5.1 Conclusion

UniAlloc was developed to replace the ad-hoc, spreadsheet-based management of lecturer workload and student supervision with a transparent, role-based web platform for the Dean's Office and its Departments. The system gives Deans and Department Heads real-time visibility into each Lecturer's capacity before an assignment is made, formalises cross-department and cross-faculty allocation through a structured request and appeal workflow, and gives students a reliable channel for requesting supervision that did not exist in the previous manual process.

Following the Waterfall model, the team moved from requirements gathering through system design, implementation, and testing in a structured, trackable sequence. Six core modules authentication and account management, faculty and department management, assignment management and capacity tracking, cross-boundary requests and workload appeals, student supervisor requests and progress tracking, and notifications and audit logging were implemented on a Next.js front end and a PHP/MySQL back end, unit tested individually, and integrated behind a shared authentication and audit layer.

The evaluation carried out with representative Deans, Department Heads, Lecturers, and students indicated that the completed system was well received across all six modules, with particularly strong satisfaction around real-time capacity tracking and dashboard visibility. Together, these results suggest that UniAlloc meets its original aim of making academic workload and supervision allocation faster, fairer, and easier to audit than the manual process it replaces.

5.2 Project Plan (Gantt Chart)

The project plan is organized according to the Waterfall phases and distributed across 15 weeks. The schedule below represents the timeline followed for key deliverables, aligned with standard project management planning practices.

For day-to-day tracking during development, a Kanban-style visual workflow board can be used to monitor task status and work-in progress.[12]

Task	Weeks 1–3	Weeks 4–6	Weeks 7–9	Weeks 10–12	Weeks 13–15
Requirements Gathering & Analysis	X				
System Design (Architecture, ERD, Wireframes)		X			
Database Setup & Implementation		X	X		
Backend Development (APIs, RBAC, Modules)			X	X	
Frontend Development (Dashboards, UI)				X	X
Deployment, Testing & Bug Fixing					X
Documentation	X	X	X	X	X
Final Report submission & presentation					X

Table 2: Project Plan (Gantt Chart)

5.3 Individual Contribution

No.	Name	Index Number	Individual Contribution
1	Mr. Suhan W.H.	UWU/IIT/23/011	Authentication, Account lifecycle management, Task acceptance & completion workflow, faculty/department management, responsive mobile layout, role promotions, Task & Priority Management.
2	Mr. Sandeep G.C.	UWU/IIT/23/020	Assignment management, capacity tracking, overload detection, Dark/light theme support, Analytics & Visualisation, workload dashboards.
3	Mr. Chathuranga D.L.	UWU/IIT/23/030	Cross-department/faculty requests, workload appeals, System settings panel, approval chain, search & filtering.
4	Mr. Arachchi D.C.K.	UWU/IIT/23/024	Student supervisor requests, Student self-registration, progress tracking, notifications & audit logging.

Table 3: Individual Contribution

References

[1] University College London. 2025. Workload Allocation Framework for Academic Staff. Retrieved May 2025 from *https://www.ucl.ac.uk/human-resources/workload-allocation-framework-academic-staff*

[2] SDGP.LK. 2025. SDGP Project Page. Retrieved May 2025 from *https://www.sdgp.lk/project/2ce4736d-57c4-402d-af8f-811fedecc158*

[3] Ian Sommerville. 2011. Software Engineering, 9th ed. Pearson Education, Boston, MA, USA.

[4] W3Schools. 2025. W3Schools. Retrieved May 2025 from *https://www.w3schools.com*

[5] Bootstrap. 2025. Bootstrap v5 introduction. Retrieved May 2025 from *https://getbootstrap.com/docs/5.0/getting-started/introduction/*

[6] Wireframe.cc. 2025. Wireframe prototyping tool. Retrieved May 2025 from *https://wireframe.cc/*

[7] Bootstrap. 2025. Build fast, responsive sites with Bootstrap. Retrieved May 2025 *from* *https://getbootstrap.com/*

[8] Bootstrap Icons. 2025. Free, high-quality, open-source icon library. Retrieved May 2025 from *https://icons.getbootstrap.com/*

[9] Stack Exchange Inc. 2008–2025. Stack Overflow. Retrieved May 2025 from *https://stackoverflow.com/questions*

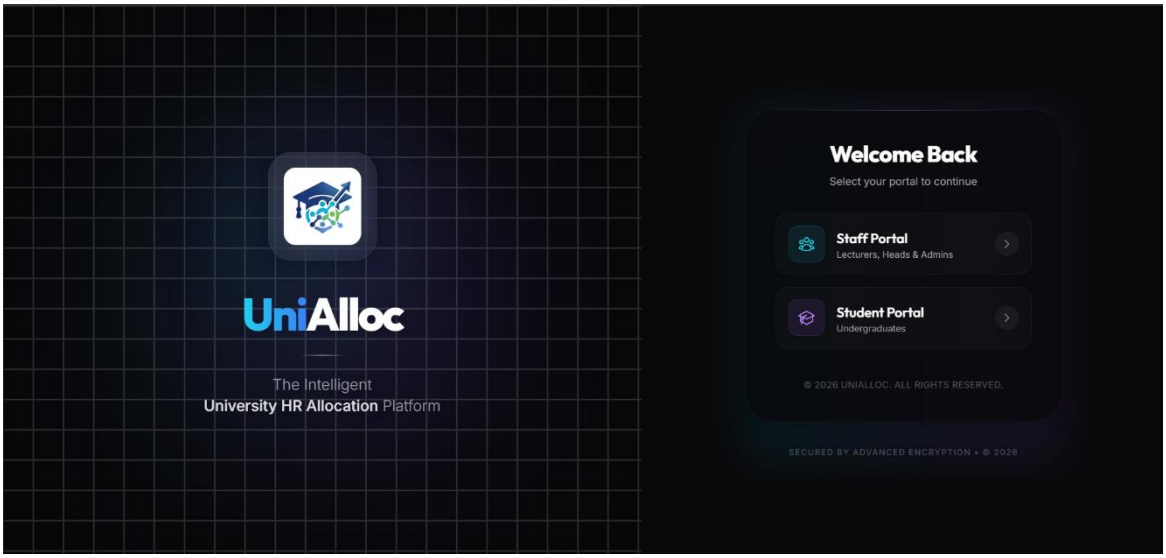
[10] PHP Group. 2025. PHP Data Objects. Retrieved May 2025 from https://www.php.net/manual/en/book.pdo.php

[11] MySQL. 2025. MySQL 8.4 Reference Manual. Retrieved May 2025 from https://dev.mysql.com/doc/refman/8.4/en/

[12] Gantt.com. 2025. How to make a Gantt chart. Retrieved May 2025 from *https://www.gantt.com/creating-gantt-charts*

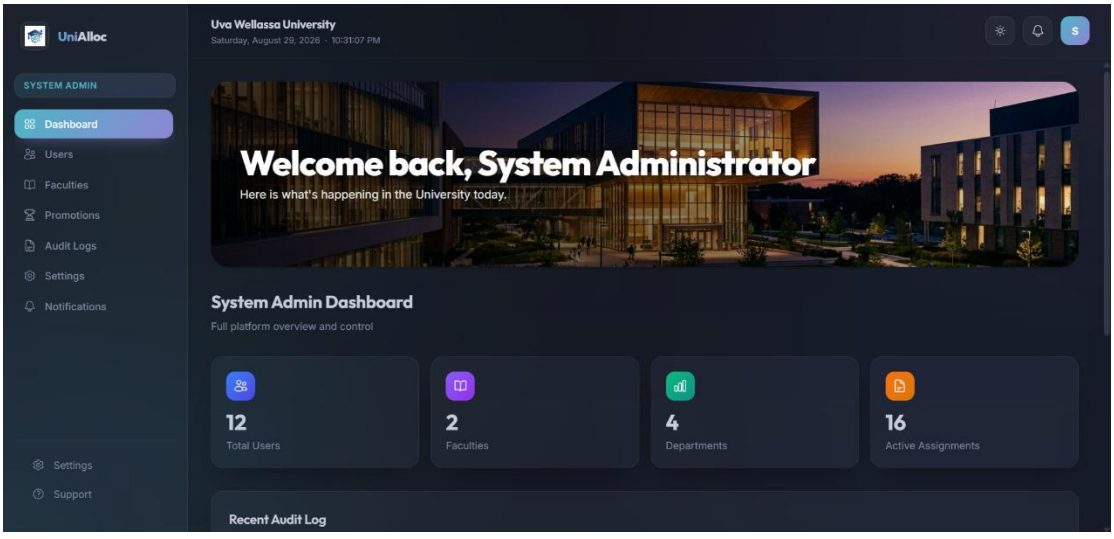
Appendixes

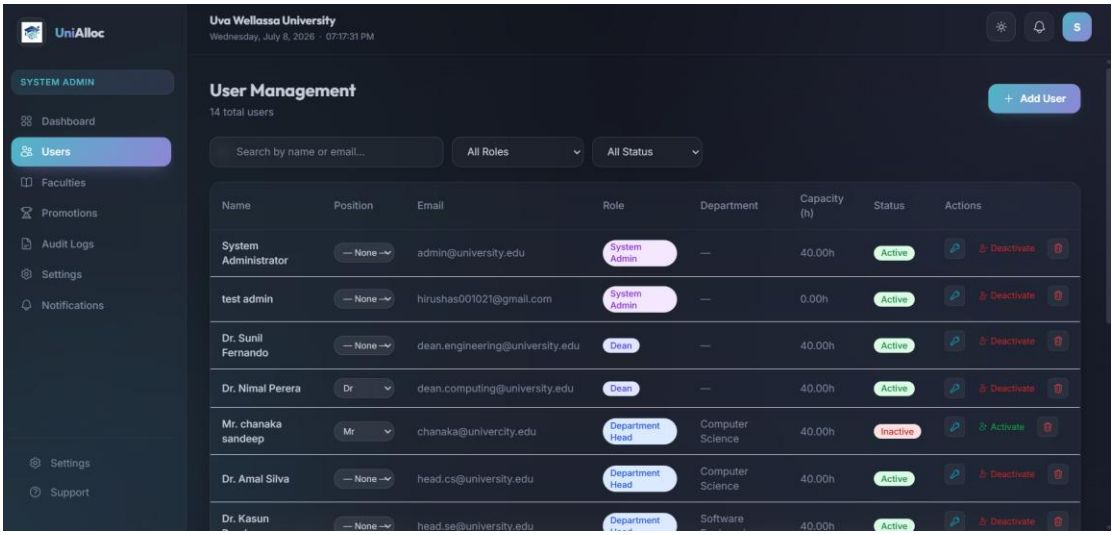
Appendix A: Login

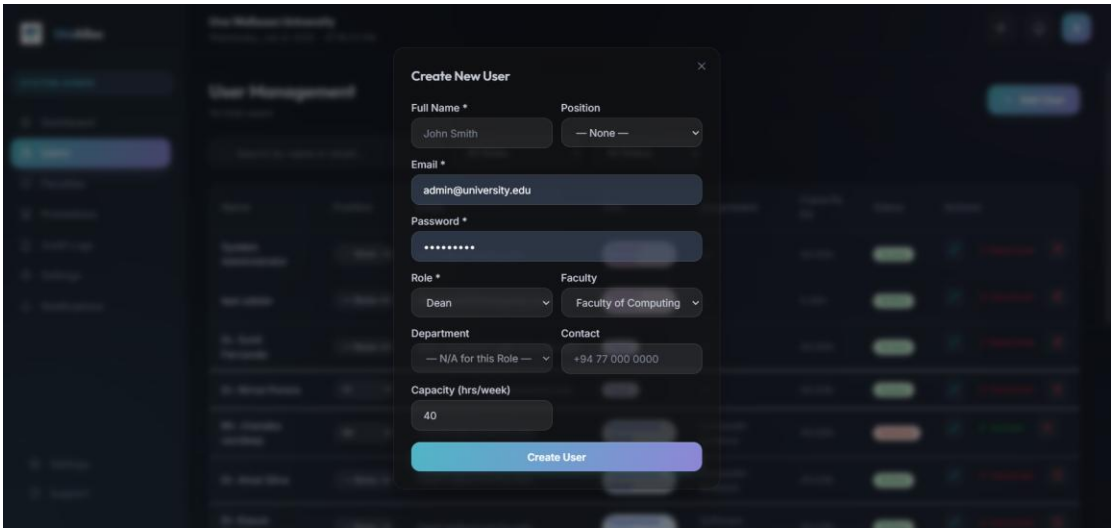


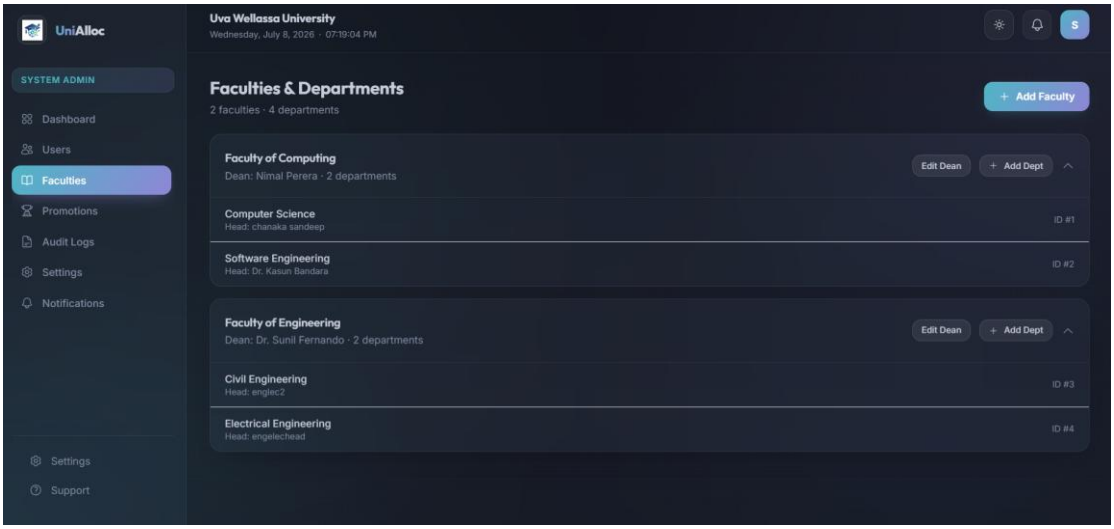
Appendix A – Login Screen

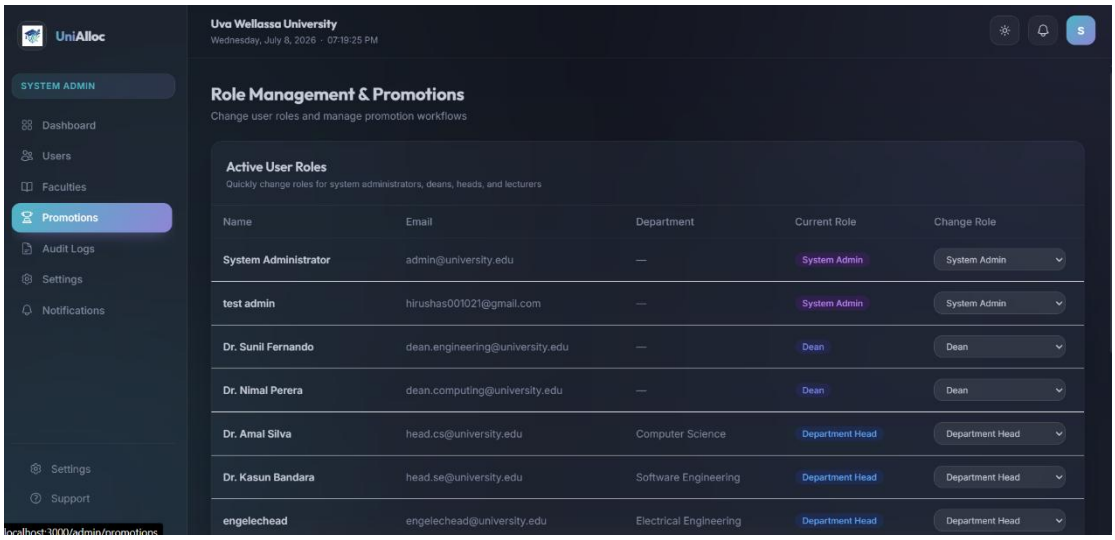
Appendix B: Admin Dashboard

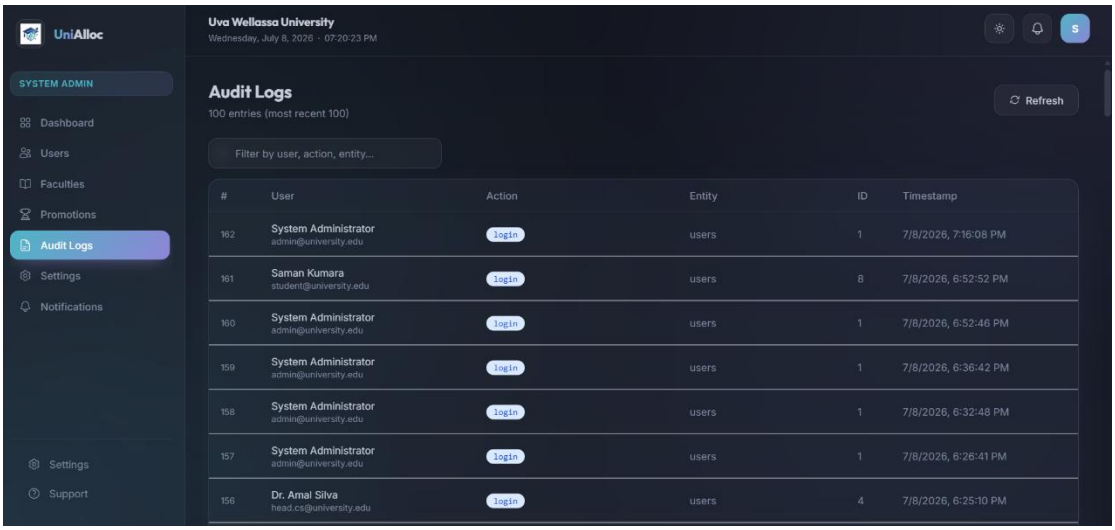


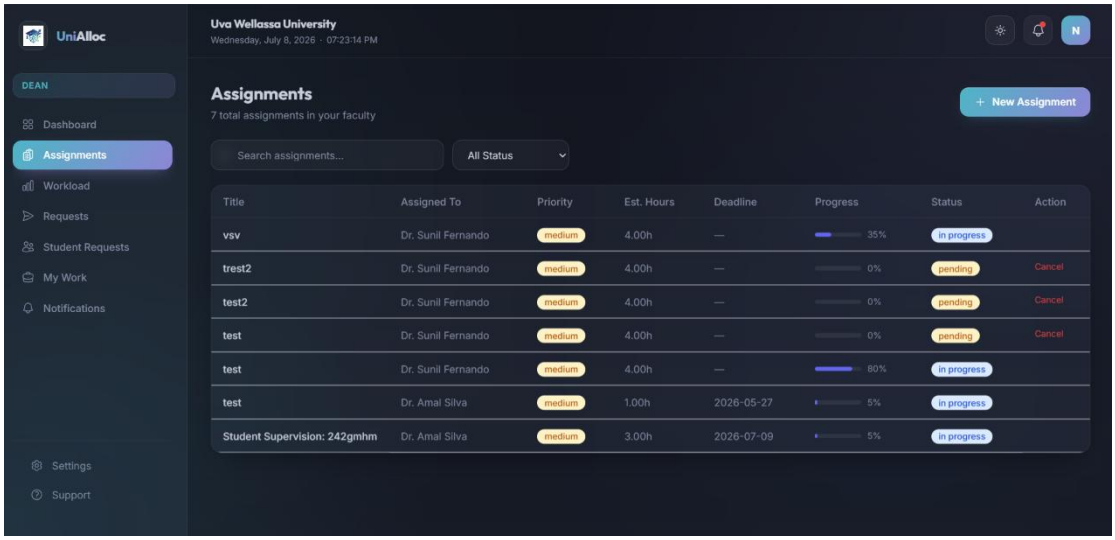

UniAlloc
Uva Wellassa University
Wednesday, July 8, 2026 · 07:17:31 PM
SYSTEM ADMIN
Dashboard
Users
Faculties
Promotions
Audit Logs
Settings
Notifications
Settings
Support
User Management
14 total users
+ Add User
Search by name or email...
All Roles
All Status
Name
Position
Email
Role
Department
Capacity (h)
Status
Actions
System Administrator
admin@university.edu
40.00h
Active
test admin
hirushas001021@gmail.com
0.00h
Active
Dr. Sunil Fernando
dean.engineering@university.edu
40.00h
Active
Dr. Nimal Perera
dean.computing@university.edu
40.00h
Active
Mr. chanaka sandeep
chanaka@univercity.edu
Computer Science
40.00h
Inactive
Dr. Amal Silva
head.cs@university.edu
Computer Science
40.00h
Active
Deactivate
Activate

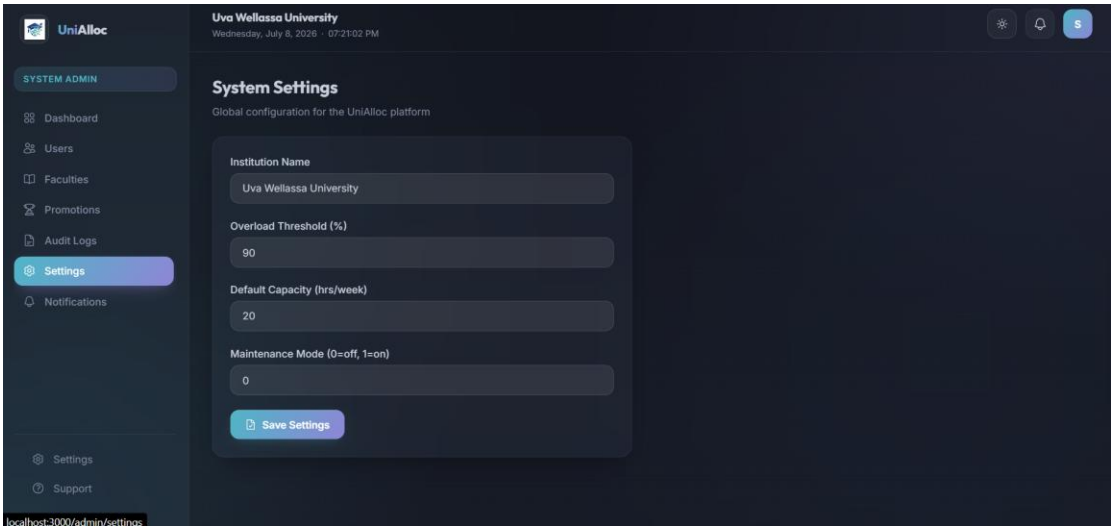

Create New User
Full Name *
John Smith
Position
— None —
Email *
admin@university.edu
Password *
Role *
Dean
Faculty
Faculty of Computing
Department
— N/A for this Role —
Contact
+94 77 000 0000
Capacity (hrs/week)
40
Create User


UniAlloc
Uva Wellassa University
Wednesday, July 8, 2026 · 07:19:04 PM
SYSTEM ADMIN
Dashboard
Users
Faculties
Promotions
Audit Logs
Settings
Notifications
Settings
Support
Faculties & Departments
2 faculties · 4 departments
+ Add Faculty
Faculty of Computing
Dean: Nimal Perera · 2 departments
Edit Dean
+ Add Dept
Computer Science
Head: chanaka sandeep
ID #1
Software Engineering
Head: Dr. Kasun Bandara
ID #2
Faculty of Engineering
Dean: Dr. Sunil Fernando · 2 departments
Edit Dean
+ Add Dept
Civil Engineering
Head: englec2
ID #3
Electrical Engineering
Head: engelechead
ID #4


UniAlloc
Uva Wellassa University
Wednesday, July 8, 2026 · 07:19:25 PM
SYSTEM ADMIN
Dashboard
Users
Faculties
Promotions
Audit Logs
Settings
Notifications
Settings
Support
Role Management & Promotions
Change user roles and manage promotion workflows
Active User Roles
Quickly change roles for system administrators, deans, heads, and lecturers
Name
Email
Department
Current Role
Change Role
System Administrator
admin@university.edu
System Admin
test admin
hirushas001021@gmail.com
System Admin
Dr. Sunil Fernando
dean.engineering@university.edu
Dean
Dr. Nimal Perera
dean.computing@university.edu
Dean
Dr. Amal Silva
head.cs@university.edu
Computer Science
Department Head
Dr. Kasun Bandara
head.se@university.edu
Software Engineering
Department Head
engelechead
engelechead@university.edu
Electrical Engineering
Department Head

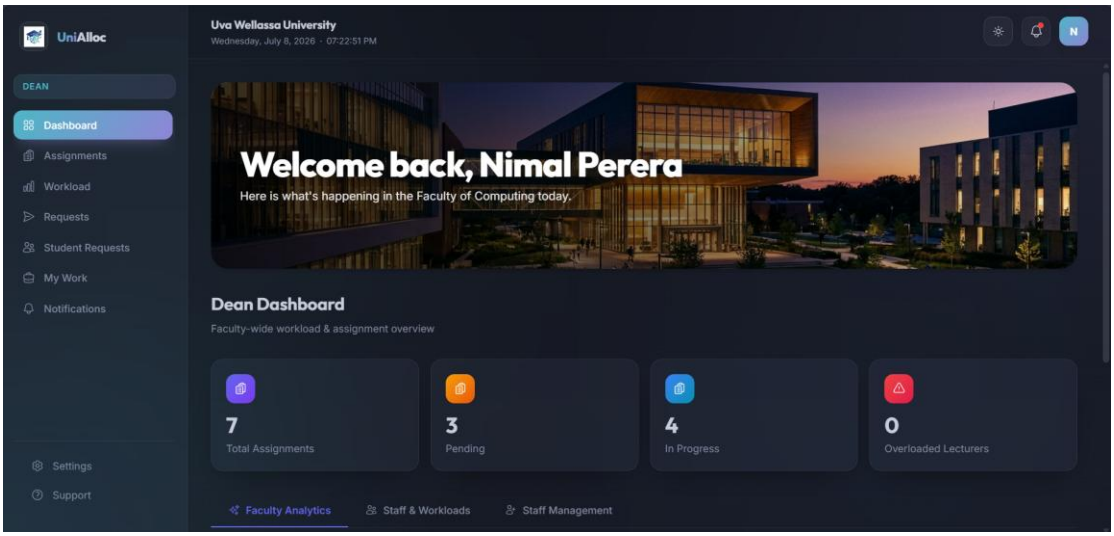

UniAlloc
Uva Wellassa University
Wednesday, July 8, 2026 · 07:20:23 PM
SYSTEM ADMIN
Dashboard
Users
Faculties
Promotions
Audit Logs
Settings
Notifications
Settings
Support
Audit Logs
100 entries (most recent 100)
Refresh
Filter by user, action, entity...
#
User
Action
Entity
ID
Timestamp
162
System Administrator
admin@university.edu
login
users
1
7/8/2026, 7:16:08 PM
161
Saman Kumara
student@university.edu
login
users
8
7/8/2026, 6:52:52 PM
160
System Administrator
admin@university.edu
login
users
1
7/8/2026, 6:52:46 PM
159
System Administrator
admin@university.edu
login
users
1
7/8/2026, 6:36:42 PM
158
System Administrator
admin@university.edu
login
users
1
7/8/2026, 6:32:48 PM
157
System Administrator
admin@university.edu
login
users
1
7/8/2026, 6:26:41 PM
156
Dr. Amal Silva
head.cs@university.edu
login
users
4
7/8/2026, 6:25:10 PM

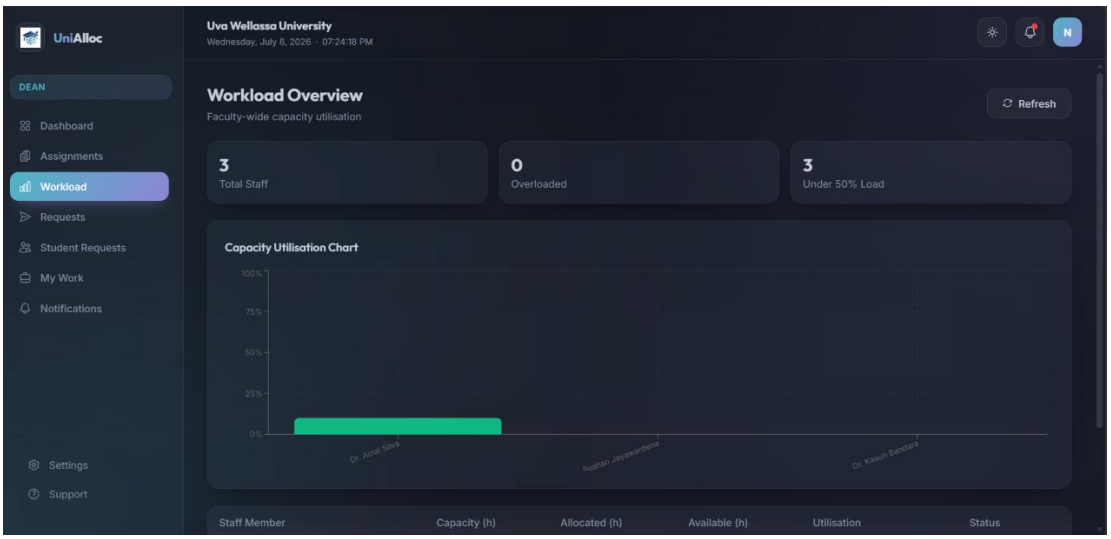

UniAlloc
Uva Wellassa University
Wednesday, July 8, 2026 · 07:23:14 PM
DEAN
Dashboard
Assignments
Workload
Requests
Student Requests
My Work
Notifications
Settings
Support
Assignments
7 total assignments in your faculty
New Assignment
Search assignments...
All Status
Title
Assigned To
Priority
Est. Hours
Deadline
Progress
Status
Action
vsv
Dr. Sunil Fernando
medium
4.00h
35%
in progress
trest2
Dr. Sunil Fernando
medium
4.00h
0%
pending
Cancel
test2
Dr. Sunil Fernando
medium
4.00h
0%
pending
Cancel
test
Dr. Sunil Fernando
medium
4.00h
0%
pending
Cancel
test
Dr. Sunil Fernando
medium
4.00h
80%
in progress
test
Dr. Amal Silva
medium
1.00h
2026-05-27
5%
in progress
Student Supervision: 242gmhm
Dr. Amal Silva
medium
3.00h
2026-07-09
5%
in progress

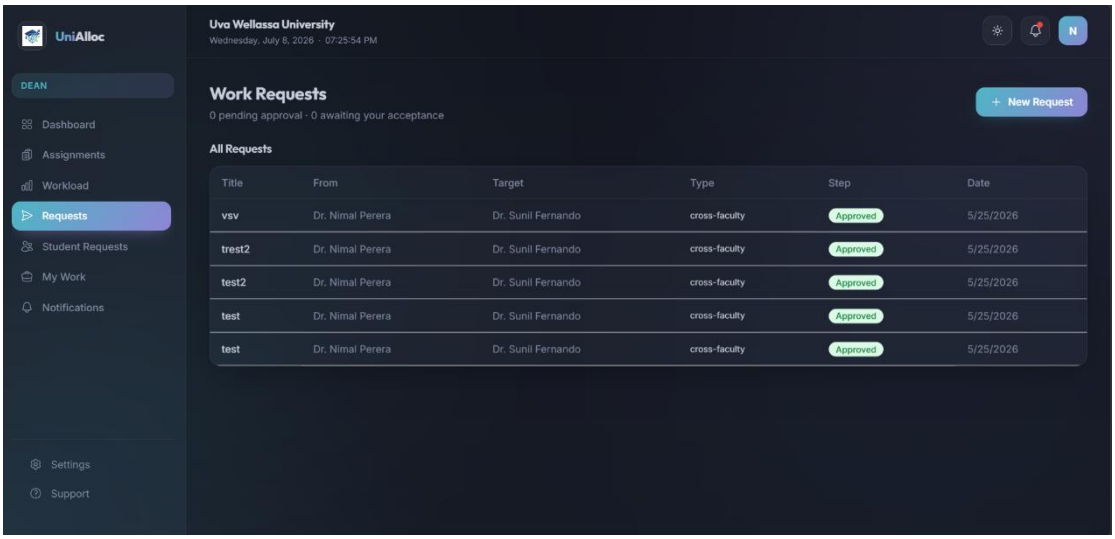


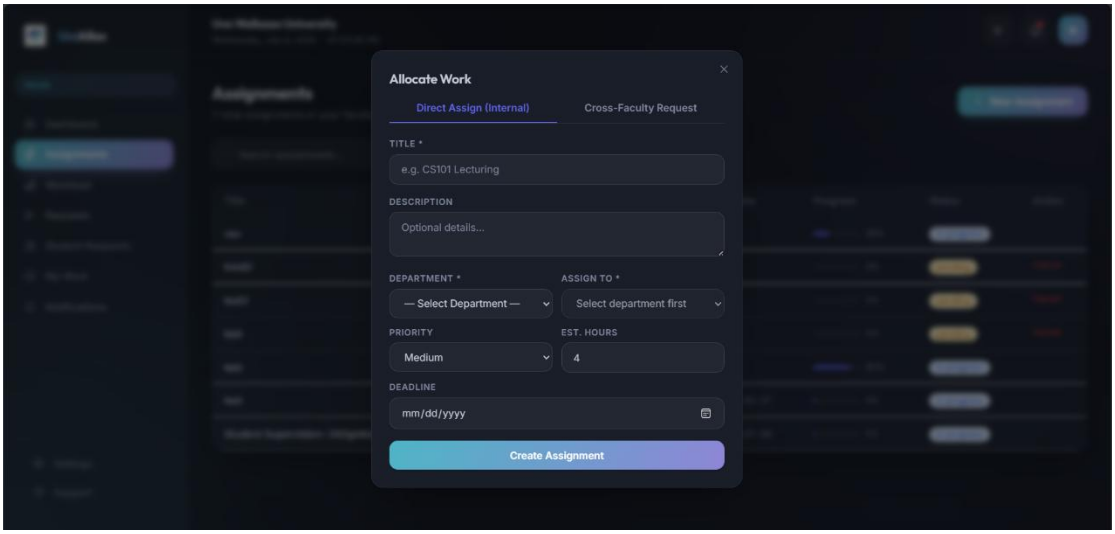
Appendix B – Admin Dashboard

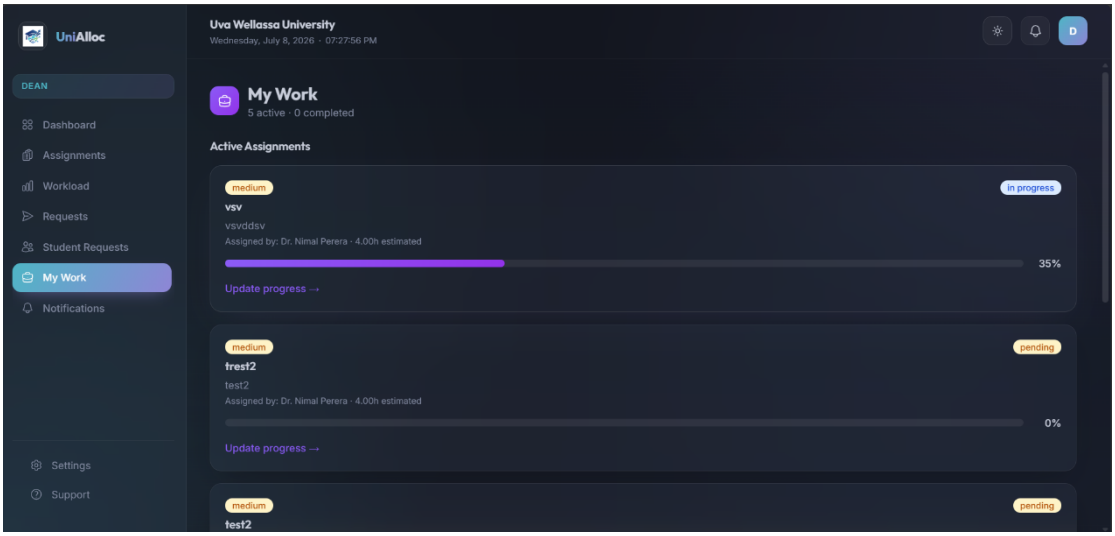
Appendix C: Dean Dashboard

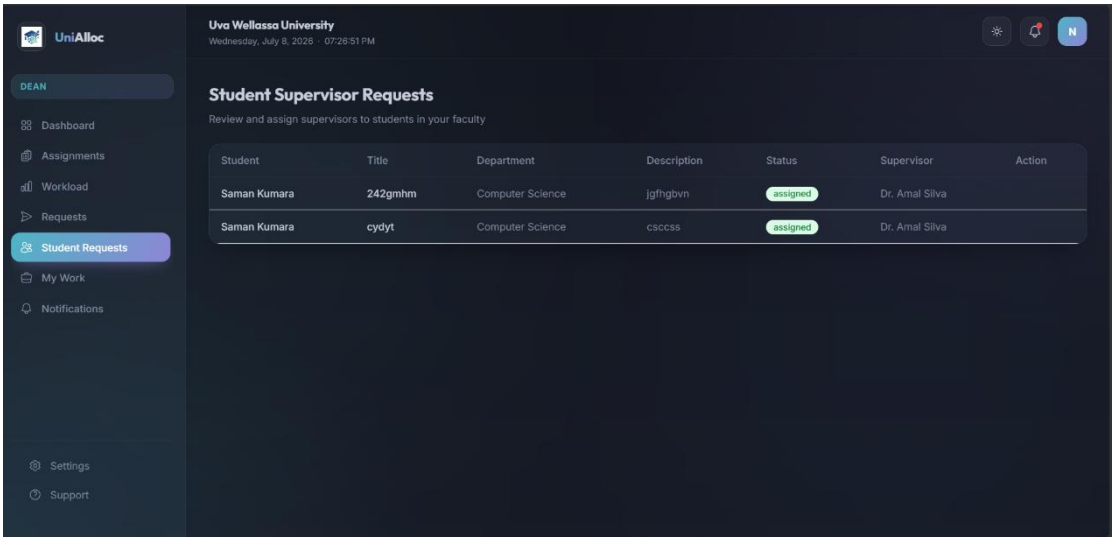





UniAlloc
Uva Wellassa University
Wednesday, July 8, 2026 · 07:25:54 PM
DEAN
Dashboard
Assignments
Workload
Requests
Student Requests
My Work
Notifications
Settings
Support
Work Requests
0 pending approval · 0 awaiting your acceptance
+ New Request
All Requests
Title From Target Type Step Date
vsv Dr. Nimal Perera Dr. Sunil Fernando cross-faculty Approved 5/25/2026
trest2 Dr. Nimal Perera Dr. Sunil Fernando cross-faculty Approved 5/25/2026
test2 Dr. Nimal Perera Dr. Sunil Fernando cross-faculty Approved 5/25/2026
test Dr. Nimal Perera Dr. Sunil Fernando cross-faculty Approved 5/25/2026
test Dr. Nimal Perera Dr. Sunil Fernando cross-faculty Approved 5/25/2026

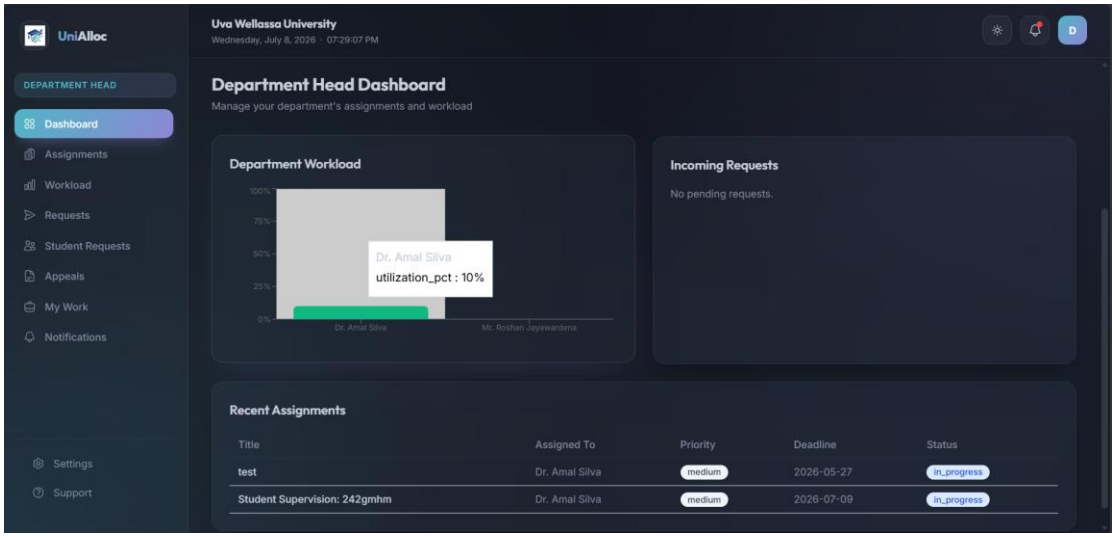

Allocate Work
Direct Assign (Internal)
Cross-Faculty Request
TITLE *
e.g. CS101 Lecturing
DESCRIPTION
Optional details...
DEPARTMENT *
— Select Department —
ASSIGN TO *
Select department first
PRIORITY
Medium
EST. HOURS
4
DEADLINE
mm/dd/yyyy
Create Assignment

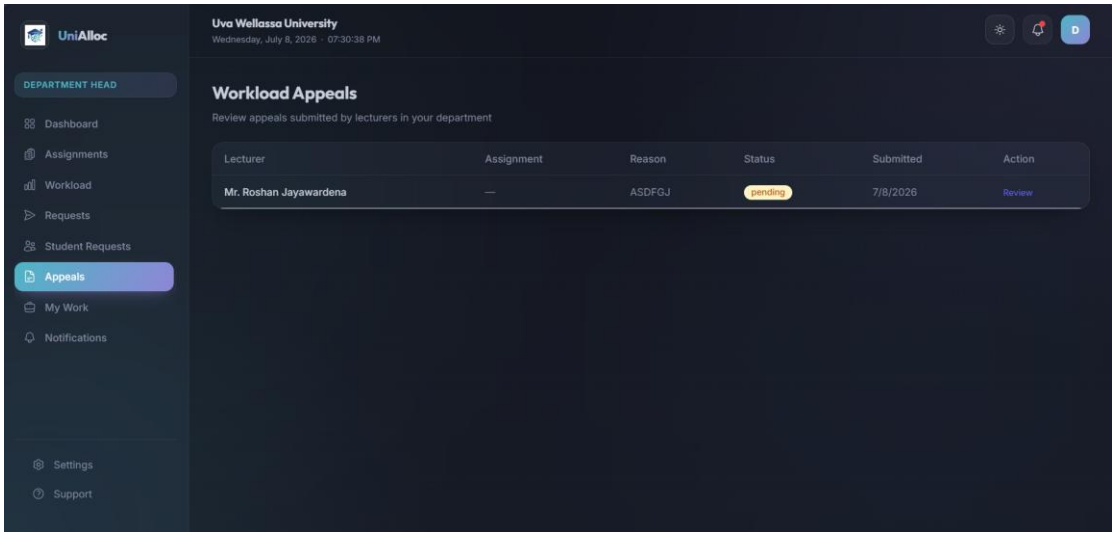

UniAlloc
Uva Wellassa University
Wednesday, July 8, 2026 · 07:27:56 PM
DEAN
Dashboard
Assignments
Workload
Requests
Student Requests
My Work
Notifications
Settings
Support
My Work
5 active · 0 completed
Active Assignments
medium
in progress
vsv
vsvddsv
Assigned by: Dr. Nimal Perera · 4.00h estimated
35%
Update progress →
medium
pending
trest2
test2
Assigned by: Dr. Nimal Perera · 4.00h estimated
0%
Update progress →
medium
pending
test2



Appendix C – Dean Dashboard

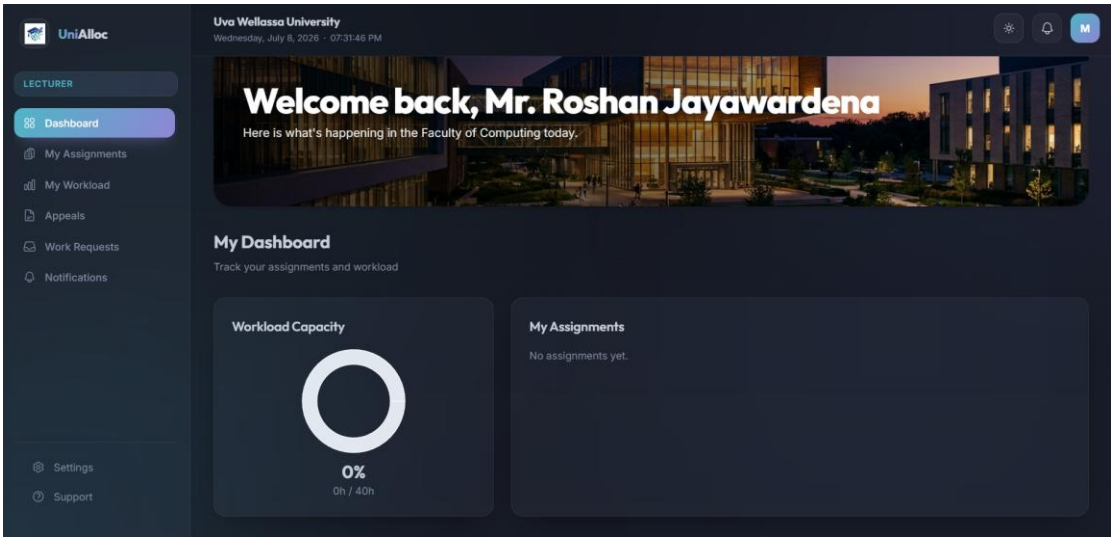
Appendix D: HOD Dashboard





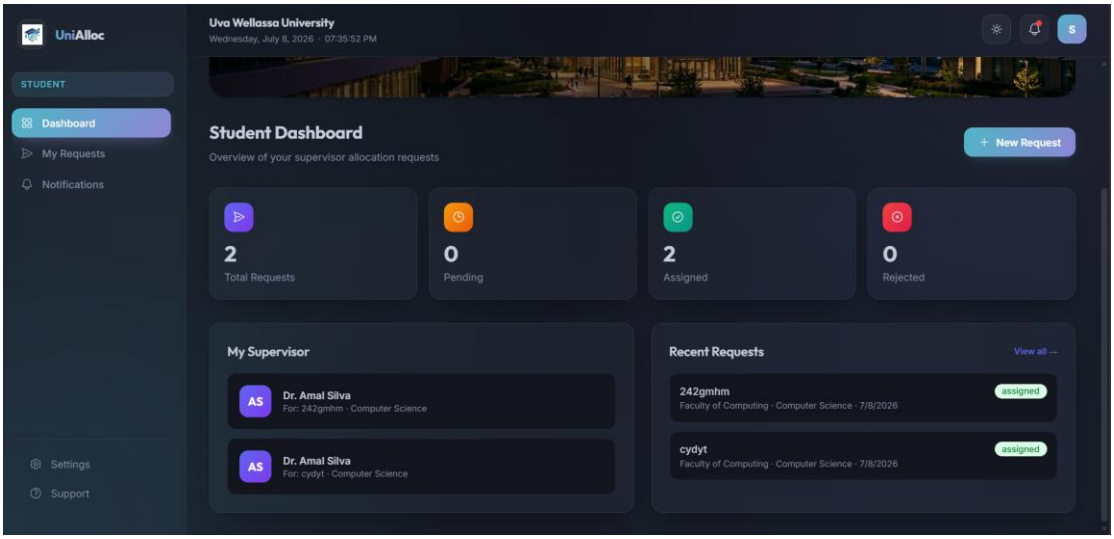
Appendix D – HOD Dashboard

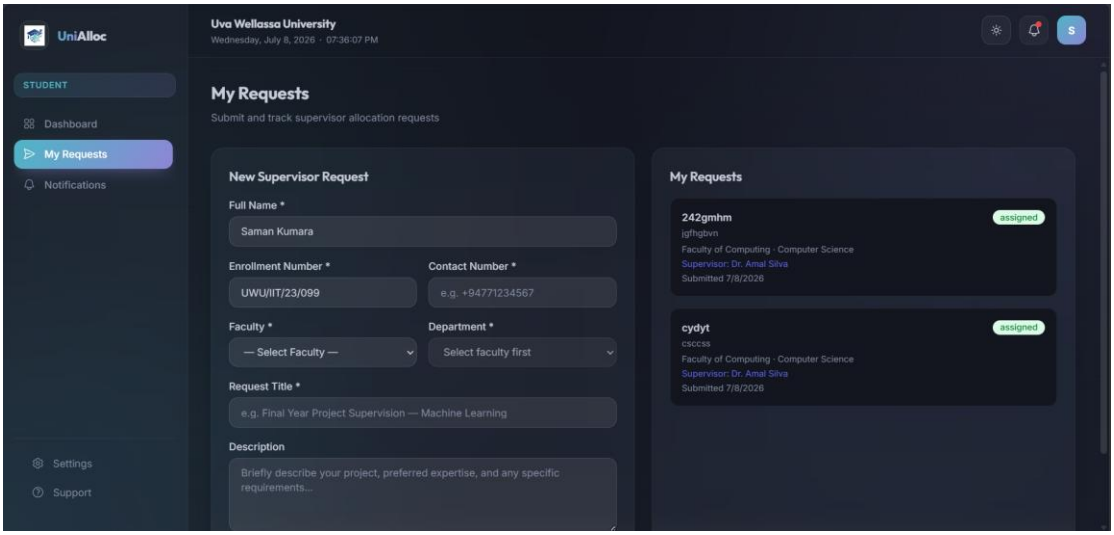
Appendix E: Lecturer Dashboard



Appendix E – Lecturer Dashboard

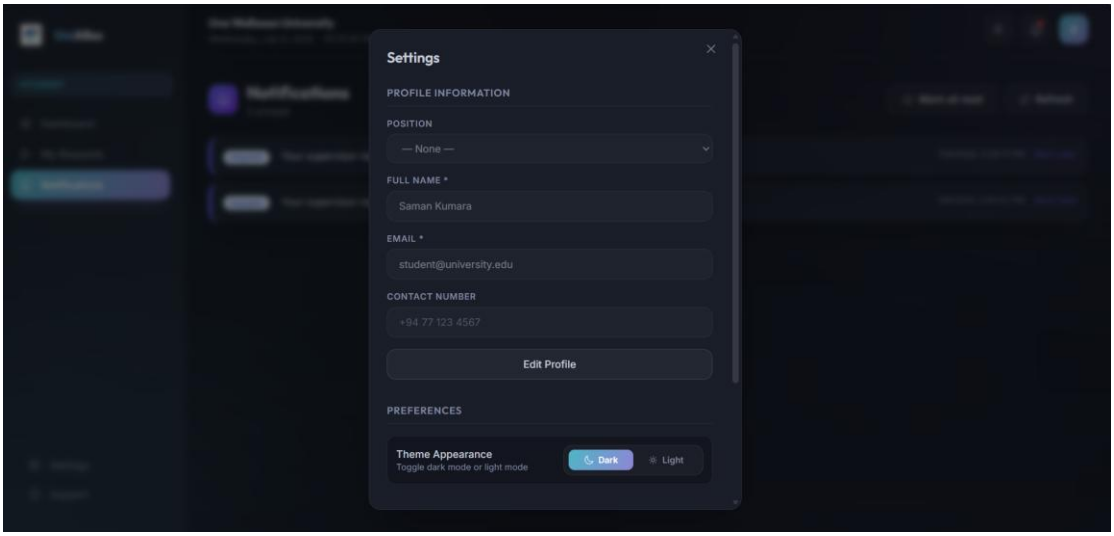
Appendix F: Student Dashboard

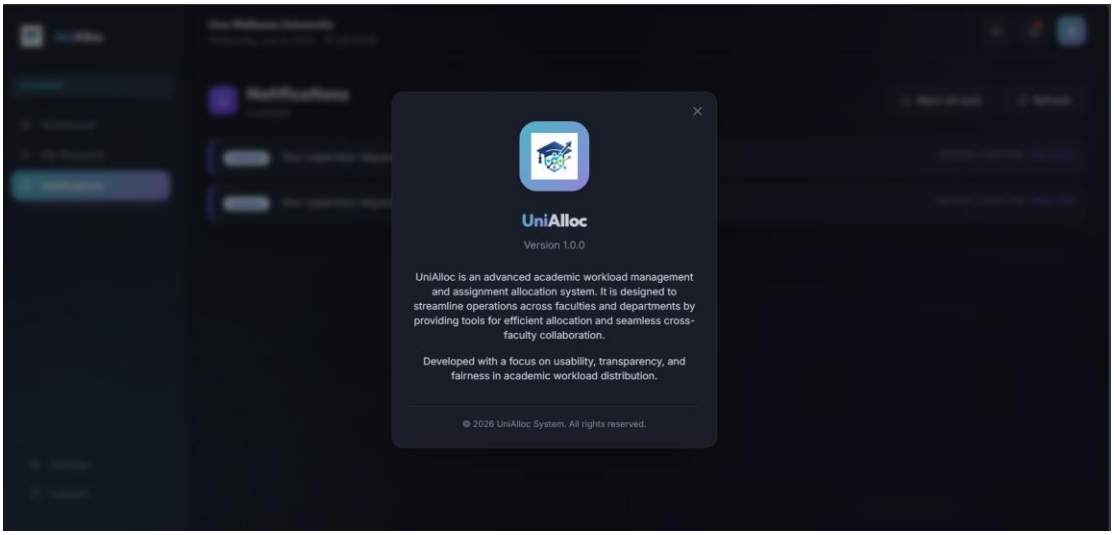


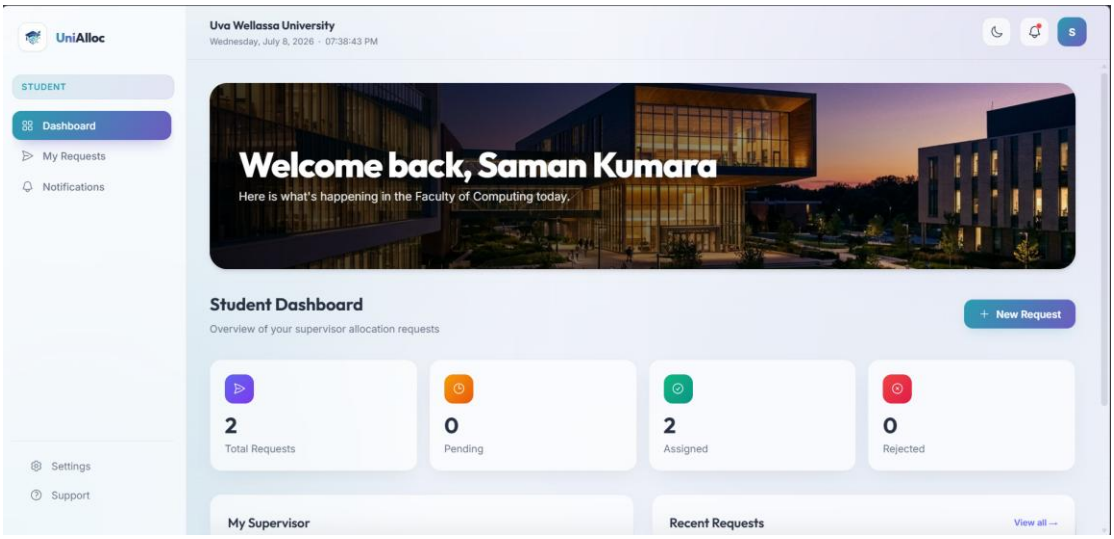


Appendix F – Student Dashboard

Appendix G: Other Common UI







Appendix G – Other Common UI

Appendix H: Test Cases

The table below lists a representative sample of the test cases used during unit and integration testing.

ID	Description	Input	Expected Result	Actual Result	Status
TC-01	Valid login	Registered email + correct password	User redirected to role-specific dashboard	As expected	Pass
TC-02	Invalid login	Registered email + wrong password	“Invalid credentials” error shown	As expected	Pass
TC-03	Add department under a faculty	Faculty selected + unique department name	Department created and listed under the faculty	As expected	Pass
TC-04	Assignment exceeding capacity	Assignment with hours that exceed Lecturer's remaining capacity	Overload warning shown before save	As expected	Pass
TC-05	Cross-department request routed correctly	Assigner selects a Lecturer outside their department	Request routed to target department's Head/Dean for approval	As expected	Pass

TC-06	Student supervisor request submission	Student submits area of interest with no preferred Lecturer	Request saved with status “Pending” and Dean notified	As expected	Pass
TC-07	Workload appeal submission	Lecturer submits appeal reason for an existing assignment	Appeal routed to relevant Department Head/Dean	As expected	Pass
TC-08	Audit log entry on role change	Admin promotes a Lecturer to Department Head	Audit log records actor, action, and timestamp	As expected	Pass

Table 4: Sample Test Cases

Appendix I: User Evaluation Questionnaire

Participants rated each statement on a 5-point Likert scale: 1 = Very Dissatisfied, 2 = Dissatisfied, 3 = Neutral, 4 = Satisfied, 5 = Very Satisfied.

1. I found the login and account management process easy to use.
2. The faculty and department structure in the system was clear and easy to navigate.
3. The assignment management and capacity/workload dashboard gave me a clear view of Lecturer workload.
4. The cross-department/faculty request and appeal process was clear and easy to follow.
5. The student supervisor request process was straightforward to use.
6. I received clear and timely notifications about actions relevant to me.
7. Overall, I am satisfied with the UniAlloc system.